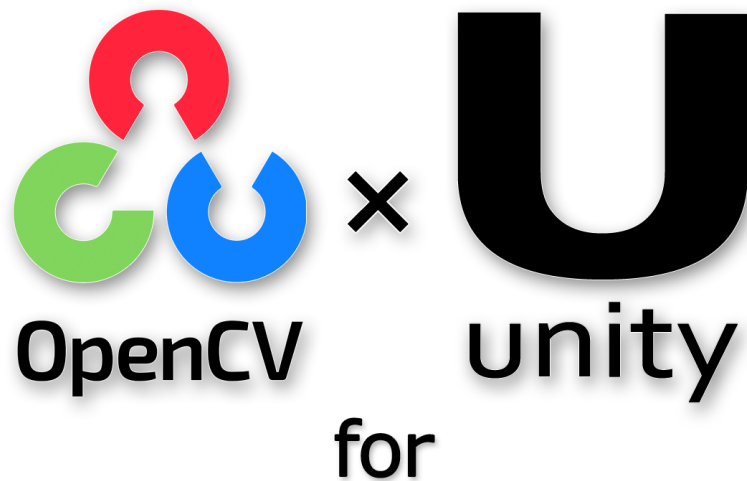


OpenCV for Unity 3.0.4



About OpenCV for Unity

OpenCV for Unity is a Unity plugin that lets you use OpenCV in Unity.

OpenCV for Unity uses *OpenCV* under *Apache 2 license*; see `Notices.txt` file in package for details.

Our Asset Features

Cross Platform

- iOS & Android & Windows10 UWP support.
- Win & Mac & Linux Standalone support.

- **WebGL** support.
- **ChromeOS** support.
- **visionOS** support. (beta)
- Support for preview in the **Editor**.

Support Latest Version

- Users can utilize **OpenCV for Unity** based on the latest version of **OpenCV (5.0.0)** for app development. (git: [opencv](#), [opencv-contrib](#))

OpenCV Java API Support

- As a **clone of OpenCV Java**, which means you can use the exact **same API as OpenCV Java 5.0.0**. [OpenCV Java documentation](#)

Easy to Use

- We provide helper functions for the interconversion of **Unity's** `Texture2D` and **OpenCV's** `Mat`. Many classes implement `IDisposable`, allowing you to manage resources using the `using` statement.

Include Many Examples

- Includes a wide variety of example usage scenarios, which consist of **scene files** and **script codes**. By running these sample applications, **you can learn how to develop OpenCV applications effectively**.
[OpenCVForUnity Examples \(GitHub\)](#)
[EnoxSoftware repositories \(GitHub\)](#)

AR VR MR

- Can be utilized for developing applications using **Augmented Reality**, **Virtual Reality**, and **Mixed Reality** technologies.

Deep Learning

- Provides support for the **dnn module**, including various frameworks such as **ONNX**, **TensorFlow**, **caffe**, **Torch**, **Darknet**, and more. See [OpenCV wiki \(GitHub\)](#)
- The **dnn module** is supported on all platforms except **UWP**. See [Support Modules](#)
- On **Unity 6** and later, the custom `MultiBackendDnn` and `MultiBackendNet` classes let you switch inference to **Unity Sentis**, enabling **GPU** inference on all platforms.

- Only **Windows** platforms can use the **CUDA backend** by following [additional steps](#).

Use of WebCamTexture

- Supports input from **Unity's** `WebCamTexture`, allowing you to perform **real-time image processing** on camera footage.

Works with many hardware

- Compatible with a **wide range of hardware gadgets** beyond just PCs and smartphones, allowing it to run on various devices.
(e.g. **HoloLens1 / HoloLens2**, **Nreal Light**, **Oculus**, **Kinect**, **RealSense**, **ZED 2** or **ZED Mini stereo camera**, and **Raspberry Pi**).

Visual Scripting Support

- You can take full advantage of all the features of **OpenCV for Unity** in **Unity's Visual Scripting**. You can learn how to integrate them through the [VisualScripting With OpenCVForUnity Example \(GitHub\)](#).

Example Code using OpenCV for Unity

AR & Recognition Examples

- [MarkerBased AR Example](#)
- [MarkerLess AR Example](#)
- [FaceMask Example](#)
- [RealTime FaceRecognition Example](#)
- [CV VTuber Example](#)

Integration Examples

- [AVPro With OpenCVForUnity Example](#)
- [Kinect With OpenCVForUnity Example](#)
- [HoloLens With OpenCVForUnity Example](#)
- [VisualScripting With OpenCVForUnity Example](#)
- [ARFoundation With OpenCVForUnity Example](#)
- [NrealLight With OpenCVForUnity Example](#)
- [Quest With OpenCVForUnity Example](#)

- [USBCamera With OpenCVForUnity Example](#)

Media & AI Examples

- [Ffmpeg With OpenCVForUnity Example](#)
 - [NativeGallery With OpenCVForUnity Example](#)
 - [YOLOv5 With OpenCVForUnity Example](#)
 - [YOLOv8 With OpenCVForUnity Example](#)
 - [KlakNDI With OpenCVForUnity Example](#)
 - [PaddleOCR With OpenCVForUnity](#)
-

Quick Links

- **Official Site** - Main product page
 - **Example Code** - GitHub repository with examples
 - **Android Demo** - Download Android demo app
 - **WebGL Demo** - Try WebGL demo online
 - **Product Introduction Video** - Watch product overview
 - **Setup Guide Video** - Step-by-step setup tutorial
 - **Forum** - Community discussions
 - **API Reference** - Complete API documentation
 - **Support Modules** - Additional module documentation
-

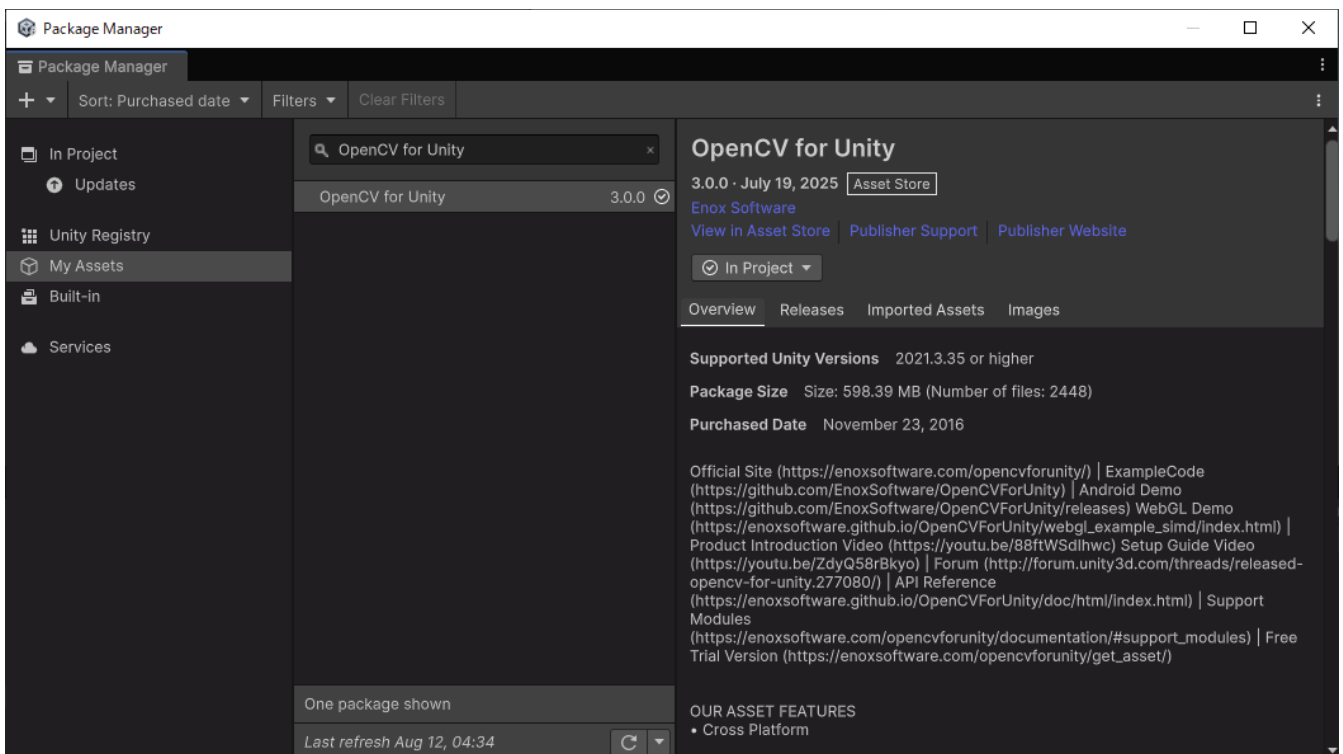
System Requirements

Platform	Requirement
Windows Standalone & Editor	Windows 8 or later
Mac Standalone & Editor	macOS 10.13 or later
Linux Standalone & Editor	Ubuntu 18.04 or later
Android	API level 24 or later
iOS	iOS Version 12.0 or later
visionOS (beta)	visionOS 1 or later

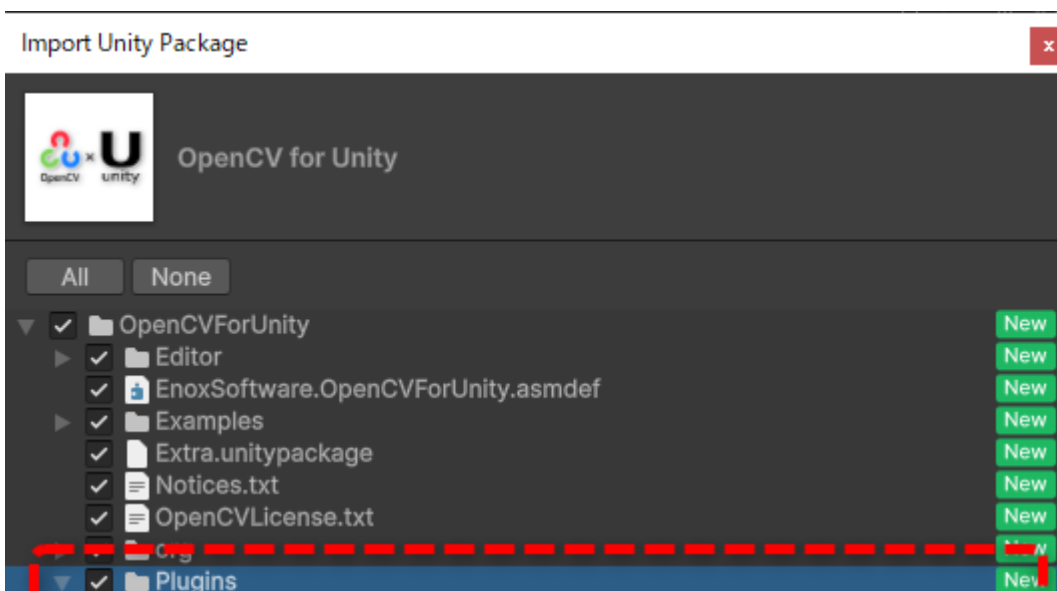
Setup Guide for Getting Started ([Setup Guide Video](#))

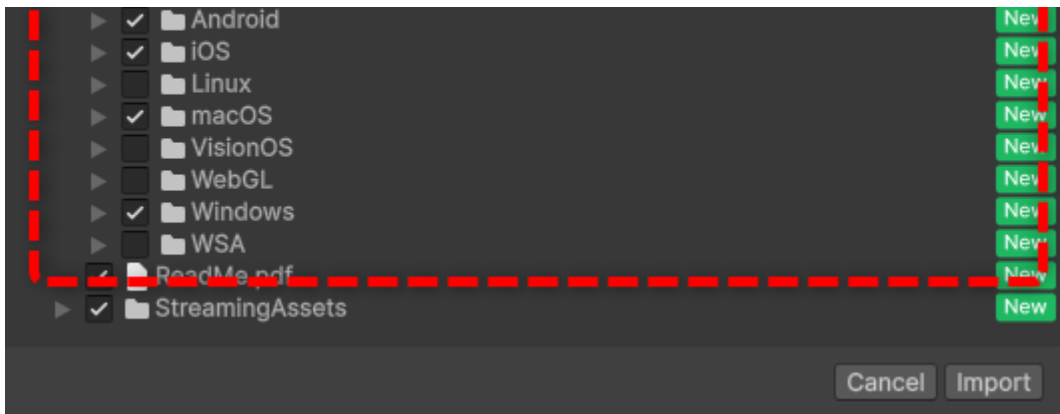
1. Import **OpenCV for Unity** from the **Package Manager** .

⚠ Important: If your project already has a previous version of the Assets/OpenCVForUnity folder, issues may occur. Please delete the Assets/OpenCVForUnity folder first before importing the new version. If you developed your project using **3.0.3**, please see [Migrating Code from 3.0.3 to the Latest Version](#) for upgrading your code.



At that time, by unchecking platforms other than those planned to be supported in your development project, unnecessary plugin files for those platforms will not be imported, reducing the project size.



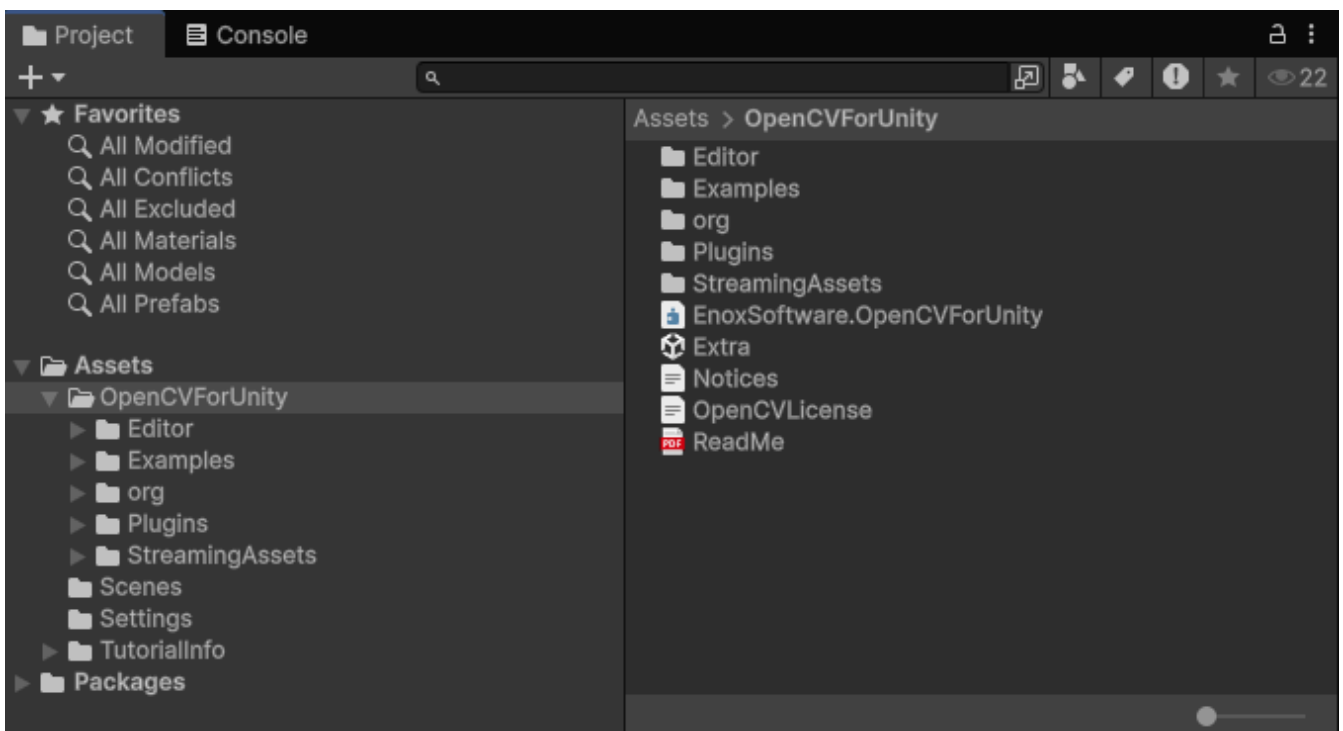


Even after import, you can easily reduce unnecessary files in the `Assets/OpenCVForUnity` folder using the `Project Size Reducer`. Particularly, by removing plugin files for platforms not planned to be supported in your development project, you can significantly reduce the project size. For details, please see [Project Size Reducer Window Description](#).

2. When the import is complete, the `Setup Tools` window will automatically appear. In the `Setup Tools` window, you can easily perform various settings. For details, please see [Setup Tools Window Description](#).

3. Setup is now complete.

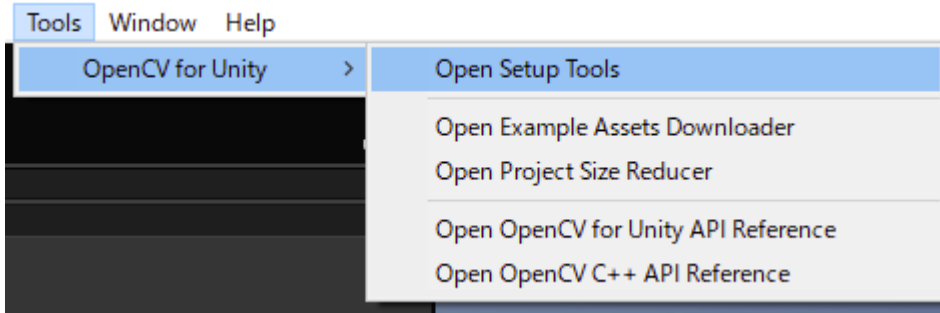
As a starting point for development using this asset, we recommend first referring to the API usage examples in the Example Scenes. Also, since the OpenCV for Unity API is the same as the OpenCV Java API, OpenCV Java tutorials are also helpful.



Example Scenes Setup Guide ([Setup Guide Video](#))

1. Complete the [Setup Guide for Getting Started](#).

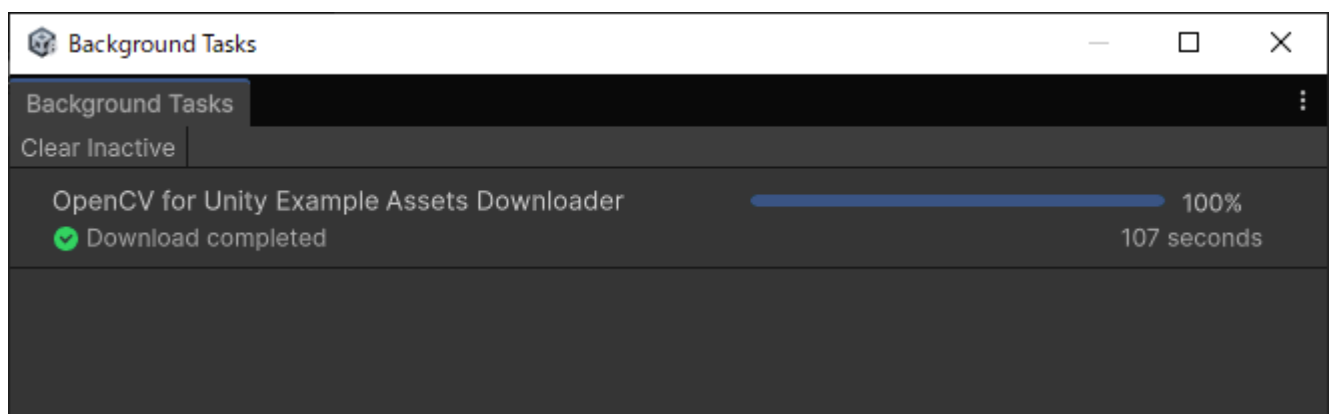
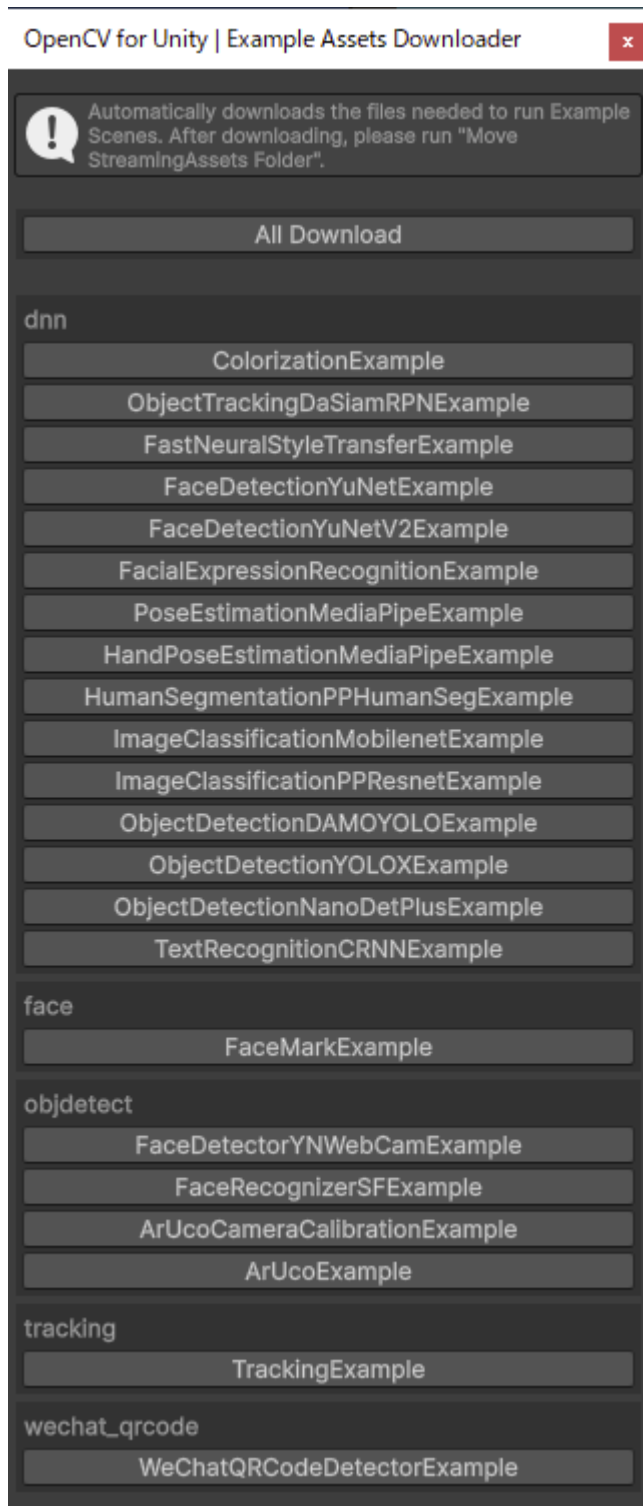
2. Select the menu item `Tools > OpenCV for Unity > Open Setup Tools` to open the `Setup Tools` window.



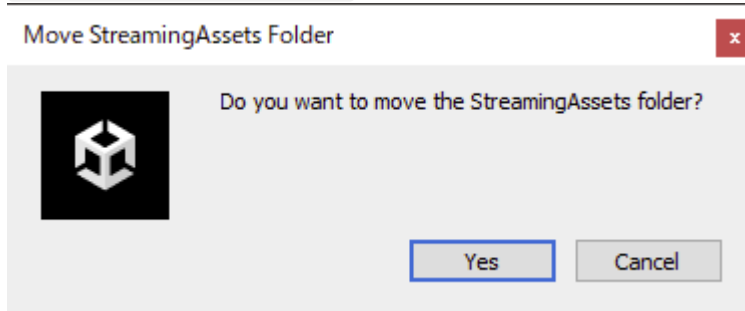
3. (**Unity 6** and later, optional) Click the `Enable Unity Sentis Integration` button in the `Setup Tools` window to enable **Unity Sentis** (`com.unity.ai.inference`) inference for **Dnn module** Example Scenes—for example when you use the `MultiBackendDnn` and `MultiBackendNet` classes / **Sentis GPU** backed path instead of the default **OpenCV CPU** backend only. This button appears only on **Unity 6** or newer; skip this step if you do not need Sentis integration.
4. Click the `Open Example Assets Downloader` button in the `Setup Tools` window to open the `Example Assets Downloader` window, then click the `All Download` button to download the necessary files for Example Scenes. [Example Assets Downloader Window Description](#)

⚠ Important: For **YOLOv5** and **YOLOv8** Example setup, please refer to the `README.md`

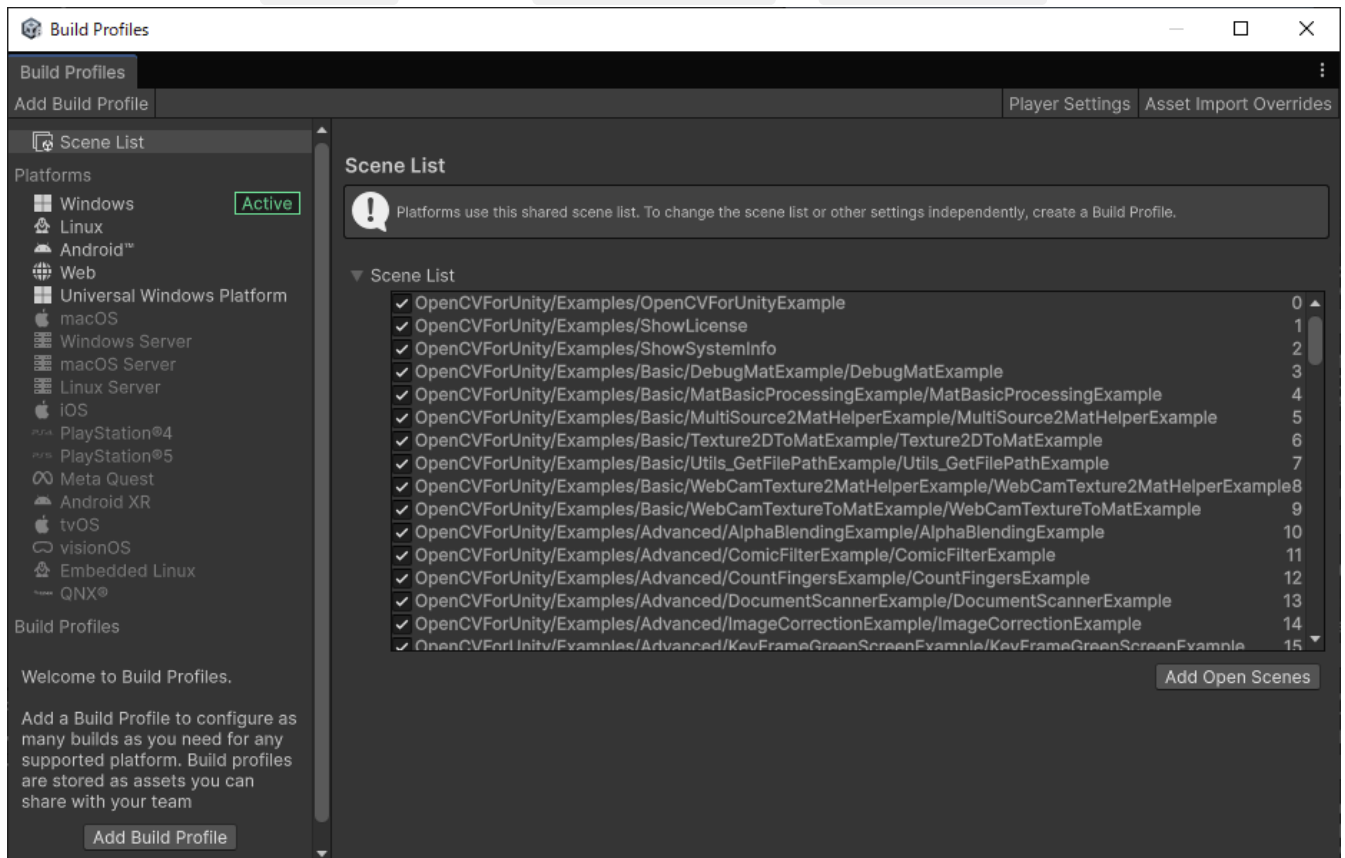
in each Example folder.



5. Click the **Move StreamingAssets Folder** button in the **Setup Tools** window to move the **Assets/OpenCVForUnity/StreamingAssets/OpenCVForUnityExamples** folder to directly under the **Assets/StreamingAssets** folder.



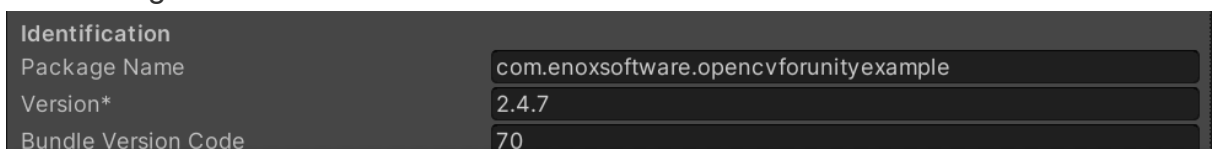
6. Click the **Add Example Scenes in Build** button in the **Setup Tools** window to add all Example Scene files in the **Examples** folder to **Scenes In Build** in **Build Settings**.

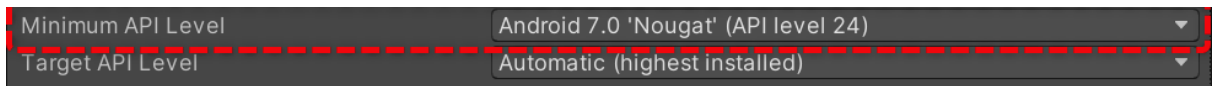


7. When building Example Scenes and running them on devices, settings may need to be changed depending on the platform.

- **Android**

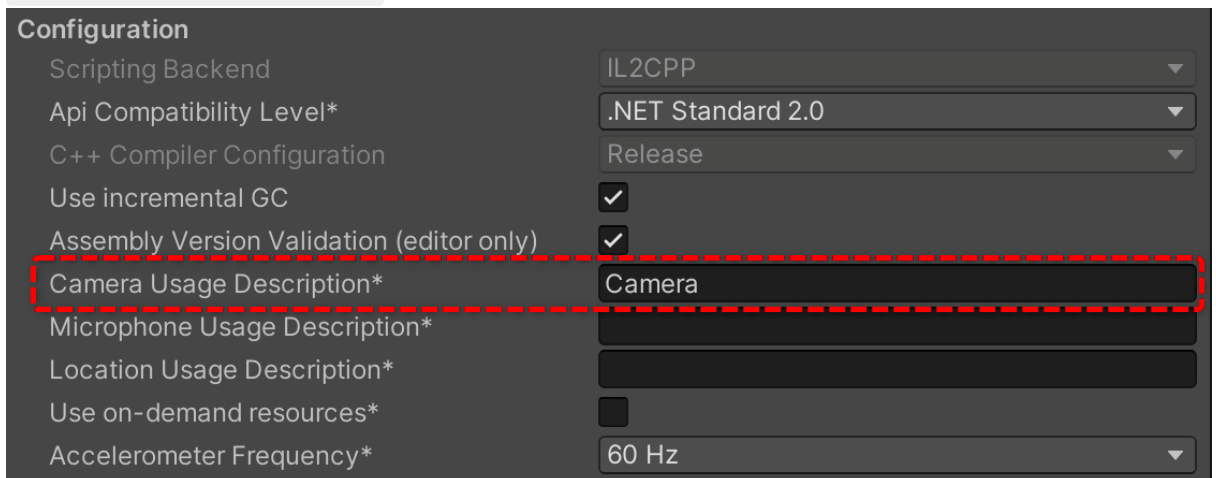
- Set **Player Settings** → **Other Settings** → **Identification** → **Minimum API Level** to **24** or higher.



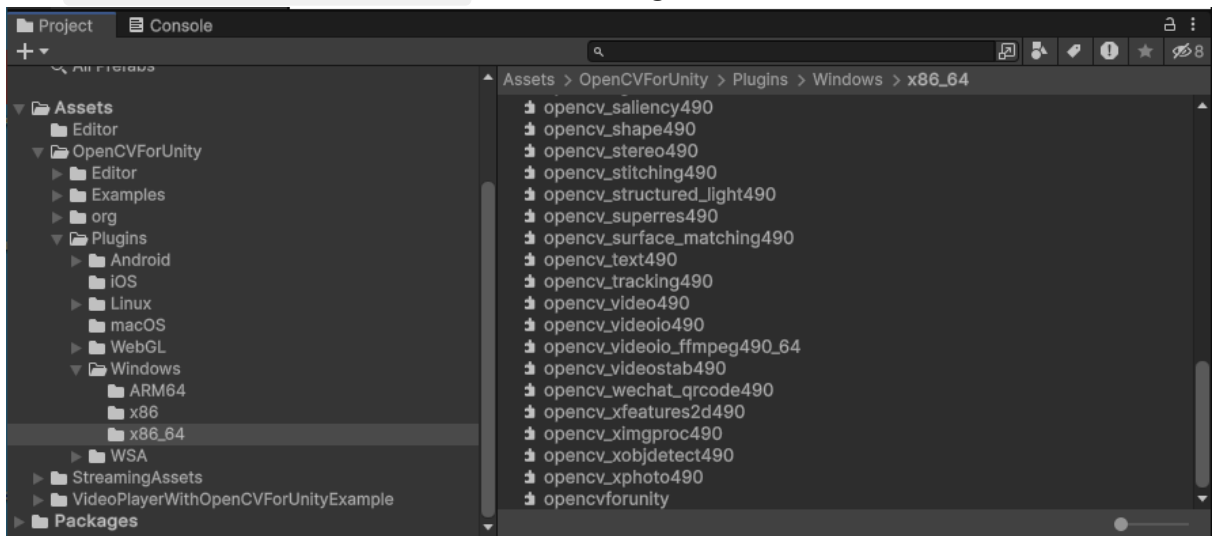


• iOS

- Set **Player Settings** → **Other Settings** → **Configuration** → **Camera Usage Description**.



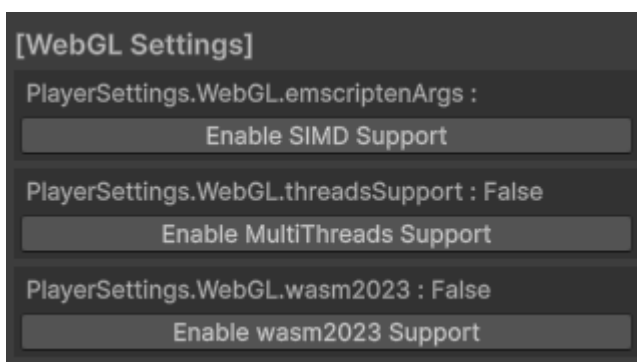
- Set **Target minimum iOS Version** to **12.0** or higher.



• WebGL

- If you want to enable multi-threading or SIMD optimization, click the **Enable MultiThreads Support** button or **Enable SIMD Support** button in the **Setup Tools** window. [Setup Tools Window Description](#)

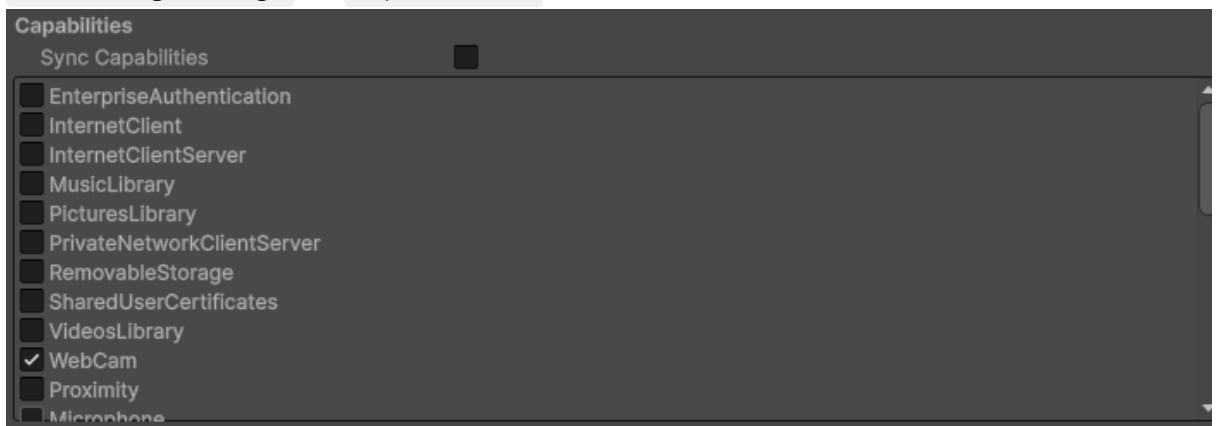
 The browser you run must support multi-threading and SIMD.



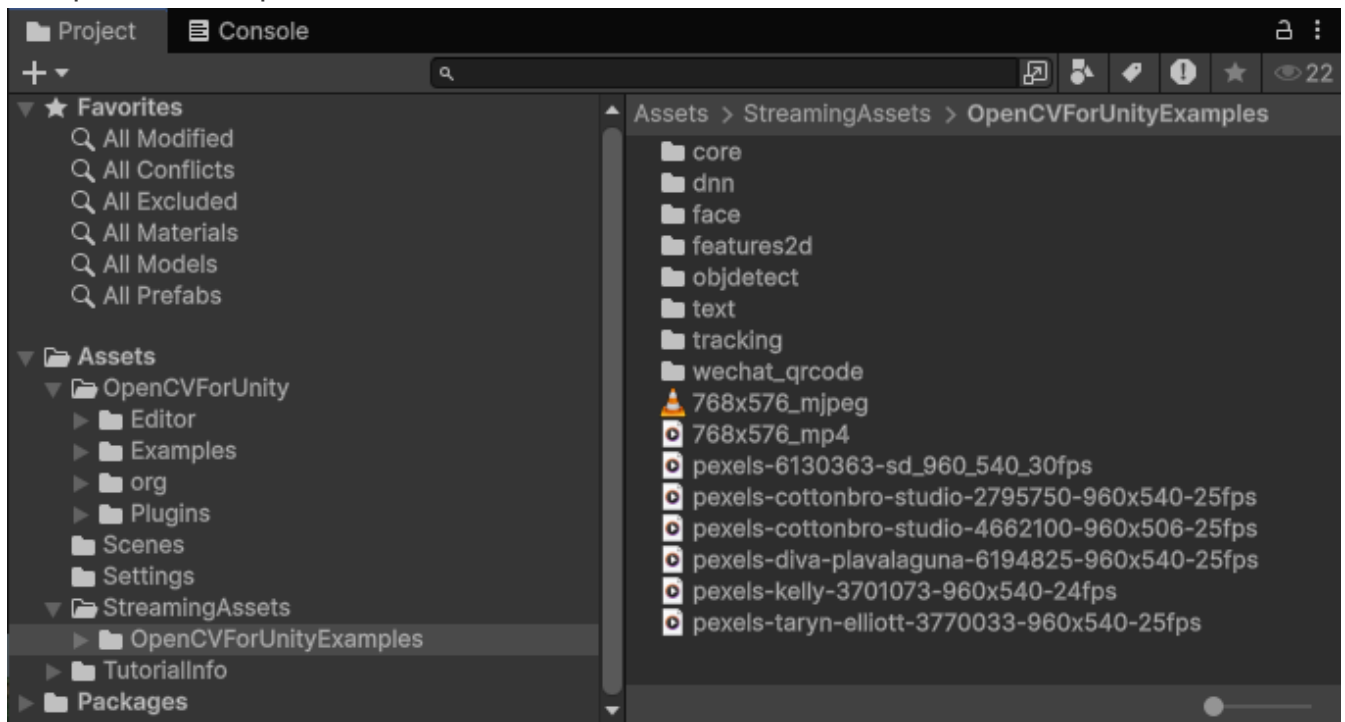
• Windows10 UWP

- If using the **WebCamTexture** class, check **WebCam** in **Player Settings** →

PublishingSettings → Capabilities .



8. Setup is now complete.



For each Example, please see [Example Scenes Description](#).

Example Scenes Description

The Example Scenes in **OpenCV for Unity** are structured to allow you to learn various **OpenCV** features progressively. They are divided into the following **4** categories, and by starting from **Basic** and progressing to **Advanced**, you can learn **OpenCV** features practically.

Basic Folder

Contains Examples related to **basic functionality and setup**:

- **Texture2DToMatExample**: Loads a sample image from `Resources` and demonstrates bidirectional conversion between Unity's `Texture2D` and OpenCV's `Mat` using

`OpenCVMatUnityUtils` , with `CvType.CV_8UC4` aligned to `TextureFormat.RGBA32`

- **WebCamTextureToMatExample**: Real-time camera example that converts live `WebCamTexture` frames to `Mat` manually without a helper class, including device enumeration, mobile permission handling, and preview display via `OpenCVMatUnityUtils`
- **WebCamTextureToMatHelperExample**: Streams live webcam frames into OpenCV `Mat` buffers using `WebCamTextureToMatHelper` , with `Inspector` configuration, helper lifecycle events (`OnFrameMatUpdated` , `OnFrameMatLayoutChanged`), and `SourceToMatControlPanel` for transport and camera UI
- **MultiSourceToMatHelperExample**: Multi-source input example using `MultiSourceToMatHelper` to switch between webcam, video file, image file, and GPU readback at runtime via `SourceToMatControlPanel` , with derived frame registration and `Texture2D` or `RenderTexture` preview output
- **MatBasicProcessingExample**: Interactive reference for fundamental `Mat` operations where each button runs a demo and displays sample code, covering `Mat` creation, shape views, channel layout, element access, and matrix math including multi-dimensional arrays
- **DebugMatExample**: Interactive playground for inspecting `Mat` data through demos including Haar-cascade face detection, video file playback, **CSRT** object tracking, numeric pixel dumps via `DebugMat` , and OpenCV exception handling with sample code display
- **Utils_GetFilePathExample**: Utility example demonstrating synchronous, coroutine, async, and **Task**-based file path resolution from `StreamingAssets` using `OpenCVForUnityEnv` , including multi-file lookup, timeout handling, and platform-specific behavior on **WebGL**

MainModules Folder

Contains Examples related to **OpenCV's main modules**:

- `core` : Basic functionality
 - **PCAExample**: Finds contour orientations by applying **Principal Component Analysis** to each contour point set, including Otsu binarization, contour detection, and drawing the principal axis on qualifying contours
 - **KMeansClusteringExample**: Segments a still image into **k** color clusters using `Core.kmeans` on flattened **RGB** pixels, demonstrating texture-to-`Mat` conversion and label-to-centroid color replacement
- `imgproc` : Image processing
 - **ThresholdExample**: Binarizes a grayscale chessboard image using `Imgproc.threshold` with **Otsu's** automatic threshold (`THRESH_OTSU`) after loading a texture into a `Mat`
 - **DrawingExample**: Demonstrates basic vector drawing primitives and **Hershey** font variants, including lines, rectangles, circles, arrowed lines, ellipses, and text with multiple font faces
 - **HoughLinesPExample**: Detects line segments in a chessboard image using **Canny**

edge detection preprocessing and probabilistic **Hough transform** (`Imgproc.HoughLinesP`) with segment overlay

- **CircleDetectionExample**: Detects circles in input frames using the **Hough gradient** method (`Imgproc.HoughCircles`) with grayscale conversion and circle overlay on the live preview via `MultiSourceToMatHelper`
- **ConnectedComponentsExample**: Labels connected regions in a binary image with `Imgproc.connectedComponentsWithStats` , visualizing per-label colors, bounding boxes, centroids, and label indices
- **MatchTemplateExample**: Locates a template patch within a source image using `Imgproc.matchTemplate` with normalized cross-correlation (`TM_CCOEFF_NORMED`) and thresholded match rectangles
- **GrabCutExample**: Segments foreground from background using an initial trimap mask and `Imgproc.grabCut` refinement (`GC_INIT_WITH_MASK`), copying foreground pixels via a binary mask
- **WrapPerspectiveExample**: Warps an image by a perspective transform defined by four source and destination corners using `Imgproc.getPerspectiveTransform` and `Imgproc.warpPerspective`
- `geometry` : Geometry processing
 - **ConvexHullExample**: Computes and draws the **convex hull** enclosing a set of random 2D points using `Core.randu` , `Imgproc.convexHull` , and `Imgproc.drawContours`
 - **MatchShapesExample**: Compares contour shapes using **Hu-moment**-based distance with `Imgproc.matchShapes` (`CV_CONTOURS_MATCH_I1`), annotating each contour with its match distance and centroid
- `features` : Feature point detection
 - **FeatureMatchingExample**: Matches descriptors between two images using **SIFT+FLANN** or **AKAZE+BruteForce-Hamming** pipelines with **k-NN** matching, Lowe's ratio test, and side-by-side match visualization
 - **SimpleBlobExample**: Detects blob keypoints in a grayscale image using configurable `SimpleBlobDetector_Params` thresholds and shape filters, visualizing results with `Features.drawKeypoints`
 - **MSERExample**: Detects stable extremal regions (**MSER**) in a still image with tunable delta and area parameters, overlaying detected region contours and bounding boxes
 - **HomographyToFindAKnownObjectExample**: Locates a known planar object in a scene image using **SIFT** feature matching, **FLANN k-NN** with Lowe's ratio test, `Geometry.findHomography` (**RANSAC**), and `Core.perspectiveTransform`
- `objdetect` : Object detection
 - **ArUcoExample**: Marker-based **AR** example that detects **ArUco** markers, **GridBoard**, **ChArUco** boards, and diamond markers from input frames, estimating camera pose with `Calib3d.solvePnP` and optional 3D cube overlay

- **ArUcoImageExample**: Detects **ArUco** markers in a static image loaded from `Resources`, estimates camera pose from marker corners, and places an **AR** cube using estimated camera parameters
- **ArUcoCameraCalibrationExample**: Interactive camera calibration using chessboard, circle grid, or **ChArUco** patterns, collecting frames via `MultiSourceToMatHelper` and saving `CameraParameters` to **XML**
- **ArUcoCreateMarkerExample**: Generates printable **ArUco** markers, boards, **ChArUco** boards, and diamond markers with dictionary selection and saves generated images via `Imgcodecs.imwrite`
- **FaceDetectionExample**: Detects frontal faces in input frames using **Haar** or **LBP** cascade classifiers with grayscale conversion, histogram equalization, and runtime cascade switching via `MultiSourceToMatHelper`
- **FaceDetectionImageExample**: Detects faces and eyes in a static image using two cascade classifiers, performing nested eye detection only inside each face **ROI**
- **AsynchronousFaceDetectionExample**: Combines background **Task**-based **Haar** detection with main-thread **LBP** refinement and **BYTETracker** multi-object tracking for responsive face detection across frames
- **FaceDetectorYNExample**: Detects faces with five landmarks using the **YuNet DNN** model via `FaceDetectorYN`, with aspect-ratio-preserving resize and optional face blurring or bounding-box visualization
- **FaceIdentificationEstimatorExample**: Detects faces, registers identities by touch, and recognizes registered faces using a **YuNet** + **SFace DNN** pipeline with sync/async inference and touch-to-register workflow
- **FaceRecognizerSFExample**: Compares face identity between a sample image and two reference images using `FaceRecognizerSF` **SFace** features with alignment, feature extraction, and cosine/**L2** matching thresholds
- **QRCodeDetectorExample**: Detects and decodes **QR codes** in input frames using `QRCodeDetector.detectAndDecodeMulti` with grayscale preprocessing, corner polygons, decoded text overlay, and `DebugMat` preview
- **QRCodeEncoderExample**: Generates a **QR code** image from user text using `QRCodeEncoder.create`, upscales the binary code for display, and optionally saves the result with `Imgcodecs.imwrite`
- **BarcodeDetectorExample**: Detects and decodes barcodes in input frames using `BarcodeDetector.detectAndDecodeWithType` for simultaneous detection, decode, and symbology identification
- **HOGDescriptorExample**: Detects people on each input frame using the default **HOG+SVM** people detector with multi-scale detection and bounding-box visualization via `MultiSourceToMatHelper`
- `dnn` : Deep learning

- **ObjectDetectionYOLOXExample**: Real-time **COCO** object detection using **YOLOX** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async inference via `MatSingleFlightSyncAsyncRunner`, and bounding-box visualization
- **ObjectDetectionDAMOYOLOExample**: Real-time **COCO** object detection using **DAMO-YOLO** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async detection, and output `Mat` visualization
- **ObjectDetectionNanoDetPlusExample**: Real-time **COCO** object detection using **NanoDetPlus** with optional **Unity Sentis** or **OpenCV DNN** frameworks, configurable simplified normalization, and async detection
- **YOLOv5ObjectDetectionExample**: Real-time **COCO** object detection using **YOLOv5** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async detection via `MatSingleFlightSyncAsyncRunner`, and output `Mat` visualization
- **YOLOv5InstanceSegmentationExample**: Real-time **COCO** instance segmentation using **YOLOv5** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async segmentation, and mask and detection box visualization
- **YOLOv5ImageClassificationExample**: Real-time **ImageNet** classification using **YOLOv5** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async classification, and top-class label visualization
- **YOLOv8ObjectDetectionExample**: Real-time **COCO** object detection using **YOLOv8** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async detection via `MatSingleFlightSyncAsyncRunner`, and output `Mat` visualization
- **YOLOv8InstanceSegmentationExample**: Real-time **COCO** instance segmentation using **YOLOv8** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async segmentation, and mask and detection box visualization
- **YOLOv8ImageClassificationExample**: Real-time **ImageNet** classification using **YOLOv8** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async classification, and top-class label visualization
- **YOLOv8PoseEstimationExample**: Real-time human pose estimation using **YOLOv8** with optional **Unity Sentis** or **OpenCV DNN** frameworks, async pose estimation, and skeleton visualization on the output `Mat`
- **ImageClassificationMobilenetExample**: Classifies each input frame with a **MobileNetV2 ImageNet** model using **OpenCV DNN** or **Unity Sentis**, with `Image2BlobParams` preprocessing and optional async classification
- **ImageClassificationPPResnetExample**: Classifies each input frame with a **PPResNet50 ImageNet** model using **OpenCV DNN** or **Unity Sentis**, with resize, center crop, mean/std normalization, and optional async classification
- **FaceDetectionYuNetExample**: Real-time face detection with bounding boxes and five facial landmarks using the older **YuNet ONNX** model (*legacy*; prefer `FaceDetectionYuNetV2Example` for the current **YuNet** API)

- **FaceDetectionYuNetV2Example**: Real-time face detection with bounding boxes and five facial landmarks using `YuNetV2FaceDetector` with optional async inference via `MatSingleFlightSyncAsyncRunner` (*recommended*)
- **FacialExpressionRecognitionExample**: Detects faces with **YuNetV2** and classifies facial expressions on each detected face region, with optional **Unity Sentis** or **OpenCV DNN** frameworks and async two-stage inference
- **MediaPipeFaceLandmarkerExample**: Detects faces, **468** facial landmarks, blendshapes, and optional 3D pose from input frames using **MediaPipe ONNX/Sentis** models with async inference and skeleton visualization
- **MediaPipeHandLandmarkerExample**: Detects hands and **21** hand landmarks per hand from input frames using **MediaPipe** palm detection and landmark models, with optional 3D skeleton visualization and async inference
- **MediaPipePoseLandmarkerExample**: Estimates full-body pose with **33** landmarks from input frames using **MediaPipe** pose detection and landmark models, with optional 3D skeleton visualization and async inference
- **MediaPipeHolisticLandmarkerExample**: Combined full-body pose, hand, and face landmark inference from input frames using chained **MediaPipe ONNX/Sentis** models with async holistic detection and combined visualization
- **HumanSegmentationPPHumanSegExample**: Segments people on each input frame with **PP-HumanSeg** and optionally composites or hides the foreground, with **Unity Sentis** or **OpenCV DNN** inference and async segmentation
- **TextRecognitionCRNNExample**: Detects text regions with **PPOCR** and recognizes strings with **CRNN** on each input frame, with polygon detection, **CTC**-greedy recognition, and optional async inference
- **ObjectTrackingDaSiamRPNExample**: Single-object visual tracking with **DaSiamRPN** after an initial **ROI** selection (*legacy*; prefer the **TrackingAPI** `TrackerDaSiamRPN` class for new projects)
- **FastNeuralStyleTransferExample**: Applies an **ONNX** fast-neural-style model to each input frame for artistic style transfer, with **Unity Sentis** or **OpenCV DNN** inference and optional async processing
- **ColorizationExample**: Recolors a grayscale photograph using an **ONNX** colorization **DNN** in **Lab** color space, with **Unity Sentis** or **OpenCV DNN** inference and side-by-side gray vs colored preview
- `imgcodecs` : Image loading and saving
 - **ImwriteScreenCaptureExample**: Captures the Unity screen, annotates it, and saves/loads the image with `Imgcodecs.imread` and `Imgcodecs.imwrite` after converting a `Texture2D` to a `Mat`
- `video` : Video processing
 - **CamShiftExample**: Tracks a color region selected by the user using mean-shift based

CamShift with **HSV** histogram back-projection and `TextureSelector` rectangle selection on input frames

- **OpticalFlowExample**: Tracks corner features across consecutive frames using **Lucas-Kanade** optical flow (`Video.calcOpticalFlowPyrLK`) with `goodFeaturesToTrack` detection and motion vector visualization
- **TransformECCExample**: Tracks a user-selected point across frames using enhanced correlation coefficient (**ECC**) alignment (`Video.findTransformECC`) with translation, Euclidean, affine, and homography motion models
- **KalmanFilterExample**: Smooths noisy cursor measurements and visualizes predict/correct steps of a **Kalman filter** with simulated measurement noise and trajectory display
- `videoio` : Video input/output
 - **VideoCaptureExample**: Plays a video file from `StreamingAssets` with seek bar and transport controls using `VideoCapture.open` , grab/retrieve pipeline, and frame pacing timed to file **FPS**
 - **VideoCaptureCameraInputExample**: Captures live camera frames through **OpenCV** `VideoCapture` with configurable **FOURCC**, resolution, and platform-specific camera permission handling
 - **VideoWriterExample**: Records Unity screen content to a video file and plays it back with `VideoWriter` and `VideoCapture` , including **fourcc**, **FPS**, and frame size setup
 - **VideoWriterAsyncExample**: Records screen content asynchronously using `AsyncGPUReadback` to reduce main-thread stalls, with `VideoWriter` frame encoding and recorded file playback
- `photo` : Image restoration and noise removal
 - **InpaintExample**: Restores damaged image regions by propagating information from surrounding pixels using `Photo.inpaint` with the `INPAINT_NS` algorithm and a single-channel inpaint mask
 - **SeamlessCloneExample**: Blends a source patch into a destination image at a given center using **Poisson** editing with `Photo.seamlessClone` (`NORMAL_CLONE`) and a full-opacity source mask
- `stereo` : Stereo vision
 - **StereoBMExample**: Computes a disparity map from a stereo image pair using block matching (`StereoBM.compute`) and normalizes **16-bit** disparity to **8-bit** for display

ContribModules Folder

Contains Examples related to **OpenCV's additional modules**:

- `face` : Face recognition and facial feature point detection
 - **FaceRecognizerExample**: Trains a small **Eigenfaces** model on two labeled faces using `BasicFaceRecognizer` , predicts identity and confidence of a test sample, and visualizes

the result grid

- **FaceMarkExample**: Detects faces and fits **68**-point facial landmarks on input frames using **Haar** cascade detection on a downscaled grayscale frame and **LBF** facemark model fitting via `MultiSourceToMatHelper`
- `wechat_qrcode` : QR code detection
 - **WeChatQRCodeDetectorExample**: Detects and decodes **QR codes** from input frames using the **WeChat QR code** model, with **DNN**-based vs built-in detection toggle and decoded text and corner polygon overlay
- `ml` : Machine learning
 - **KNNExample**: Classifies a test point by majority vote among its **k** nearest training neighbours using `KNearest` , visualizing training points, the query point, and the **k**-th neighbour distance
 - **SVMExample**: Trains a linear **SVM** on 2D points and visualizes the decision boundary and support vectors by classifying every pixel to render decision regions
- `tracking` : Object tracking
 - **TrackingExample**: Tracks a user-selected rectangle region across consecutive input frames using multiple **OpenCV** trackers, comparing classical trackers (**KCF**, **CSRT**, **MIL**) with **DNN**-based ones (**Vit**, **DaSiamRPN**, **Nano**)
 - **LegacyTrackingExample**: Tracks a user-selected rectangle region using the deprecated `legacy_Tracker` **API**, with side-by-side comparison of **Boosting**, **CSRT**, **KCF**, **MedianFlow**, **MIL**, **MOSSE**, and **TLD** trackers
- `text` : Text detection and recognition
 - **TextDetectionExample**: Locates text regions in a scene image using the **Extremal Region (ER)** filter pipeline with **NM** channels, **ERFilter NM1/NM2** classifiers, and region grouping into bounding boxes
 - **TextRecognitionExample**: Detects text regions with **ER**-based **NM1/NM2** filters and recognizes characters using the **OCR HMM** decoder with transition/emission matrices for per-region character recognition
- `plot` : Graph drawing
 - **PlotExample**: Renders a random 1D data series as a 2D line plot using the **OpenCV** plot module (`Plot2d`), with customizable background and line colors converted to a Unity `Texture2D` for display
- `bgsegm` : Background segmentation
 - **BackgroundSubtractorExample**: Compares foreground masks produced by different background subtraction algorithms on video, switching between **KNN**, **MOG2**, **CNT**, **GMG**, **GSOC**, **LSBP**, and **MOG** subtractors at runtime via `MultiSourceToMatHelper`

Advanced Folder

Contains **advanced implementation examples**:

- **AlphaBlendingExample:** Benchmarks multiple ways to alpha-composite foreground, background, and alpha `Mat` objects on a still image, comparing seven implementations including `Core` operations, `MatBufferUtils`, and **Span** across original, **2x** upscale, and **ROI** crop sizes
- **BallTrackingBasedOnColorExample:** Tracks a colored ball in input frames using **HSV** segmentation, morphological erode/dilate, and contour analysis with `minEnclosingCircle` and moments, including tap-to-calibrate **HSV** range from a screen point
- **ComicFilterExample:** Applies a manga-style comic filter to each input frame in real time by delegating image processing to a reusable `ComicFilter` helper class with in-place `Mat` processing via `MultiSourceToMatHelper`
- **CountFingersExample:** Estimates the number of raised fingers from a hand contour using **HSV** skin-color segmentation via `ColorBlobDetector`, **convex hull**, and convexity defects, with tap-to-calibrate hand color before detection starts
- **DocumentScannerExample:** Detects a quadrilateral document in input frames using edge detection on the **Y** (luminance) channel of **YUV** color space, finds a **4**-point contour, and applies `warpPerspective` with split preview of original overlay and corrected output
- **ImageCorrectionExample:** Adjusts contrast, brightness, gamma, and optional binary threshold on each input frame using `Core.convertScaleAbs`, a **256**-entry **LUT** for gamma correction, and `Imgproc.threshold` via `MultiSourceToMatHelper`
- **KeyFrameGreenScreenExample:** Simulates a green-screen shot without a physical backdrop by capturing a still background frame on tap, differencing each new frame, and replacing background pixels with solid green (`#00FF00`) for key-frame video editing
- **MultiObjectTrackingExample:** Detects objects with **YOLOX** and assigns persistent track **IDs** using **ByteTrack**, with **OpenCV DNN** or **Unity Sentis** inference toggle and async detection pipeline via `MatSingleFlightSyncAsyncRunner`
- **MultiObjectTrackingBasedOnColorExample:** Tracks multiple colored blobs (blue, yellow, red, green) in input frames without **DNN** using fixed **HSV** ranges per color via `ColorObject` and contour moments for centroid tracking with labeled overlay
- **PhysicalGreenScreenExample:** Composites a background image over a physical green-screen subject using **HSV_FULL** range masking with `Core.inRange`, tap-to-pick key color, slider-adjusted **H/S/V** tolerance, and `copyTo` with mask
- **PolygonFilterExample:** Applies a low-poly effect by **Delaunay** triangulation and flat color fills per triangle on each input frame, using edge detection on a downscaled **GRAY** `DerivedFrame` and `Subdiv2D` triangulation with `fillConvexPoly`

Setup Tools Window Description

Select the menu item `Tools > OpenCV for Unity > Open Setup Tools` to open the `Setup Tools`

window.

 OpenCV for Unity | Setup Tools 



[Setup for Example Scenes]

Enable Unity Sentis (com.unity.ai.inference) inference in OpenCV for Unity Dnn module example scenes.

Enable Unity Sentis Integration

Automatically downloads the files needed to run Example Scenes. After downloading, please run "Move StreamingAssets Folder".

Open Example Assets Downloader

Move the files from the "OpenCVForUnity/StreamingAssets" folder to the "Assets/StreamingAssets" folder.

Move StreamingAssets Folder

Add Example Scenes to "Scenes In Build" in BuildSettings.

Add Example Scenes In Build

[Optional]

Unnecessary components can be removed to reduce the size of the project.

Open Project Size Reducer

Set the appropriate ImportSettings for the files in the Plugins folder. Please reconfigure when you change the version of UnityEditor.

Set Plugin Import Settings

Import when using plugins that exclude opencv_contrib modules to reduce build size. See ReadMe.pdf for more information.

Import Extra Package

Add "OPENCV_DONT_USE_UNSAFE_CODE" to ScriptCompilationDefines in BuildSettings.

Disable Use Unsafe Code

[WebGL Settings]

PlayerSettings.WebGL.emscriptenArgs :

Enable SIMD Support

PlayerSettings.WebGL.threadsSupport : False

Enable MultiThreads Support

PlayerSettings.WebGL.wasm2023 : True

This window is a collection of tools to make **OpenCV for Unity** project setup easier. Each section has a specific role and is structured as follows.

Setup for Example Scenes

This section contains tools for preparing and setting up files necessary to run Example Scenes.

- **Enable Unity Sentis Integration**

Shown only on **Unity 6** or later. Opens a confirmation dialog, then adds

`com.unity.ai.inference` and related `asmdef` wiring so **Dnn module** Example Scenes can use **Unity Sentis** inference (and the `MultiBackendDnn` and `MultiBackendNet` / **Sentis GPU** path) when needed.

- **Open Example Assets Downloader**

Opens the `Example Assets Downloader` window that automatically downloads files necessary to run Example Scenes. For details, please see [Example Assets Downloader Window Description](#).

After downloading, please execute the next step `Move StreamingAssets Folder`.

- **Move StreamingAssets Folder**

Moves downloaded files from the `Assets/OpenCVForUnity/StreamingAssets` folder to the `Assets/StreamingAssets` folder.

- **Add Example Scenes In Build**

Automatically adds Example Scenes to `Scenes In Build` in `Build Settings`.

Optional

This section contains convenient tools for optional use, such as project size reduction and importing `Extra` packages.

- **Open Project Size Reducer**

Opens the `Project Size Reducer` window that removes unnecessary components and reduces project size. For details, please see [Project Size Reducer Window Description](#).

- **Set Plugin Import Settings**

Sets import settings for files in the `Plugins` folder appropriately in bulk.

Please execute when changing Unity editor version or settings.

- **Import Extra Package**

Imports additional packages necessary when you want to reduce build size by excluding the `opencv_contrib` module. For details, please see [Tips for Each Platform](#).

- **Disable Use Unsafe Code**

Adds `OPENCV_DONT_USE_UNSAFE_CODE` to `ScriptCompilationDefines` in `Build Settings`. With this setting, OpenCV for Unity's internal code will not use unsafe code.

Enabling unsafe code often results in faster code, so we recommend keeping it enabled.

WebGL Settings

This section is for settings to improve performance during **WebGL** builds.

- **Enable SIMD Support**

Adds settings to enable **SIMD** (Single Instruction Multiple Data) instruction set to `PlayerSettings.WebGL.emscriptenArgs` to improve performance.

Please enable when aiming for significant speedup in WebGL builds.

- **Enable MultiThreads Support**

Enables `PlayerSettings.WebGL.threadsSupport` to enable multi-threading in WebGL.

Please enable when aiming for speedup in WebGL builds.

- **Enable wasm2023 Support**

Enables `PlayerSettings.WebGL.wasm2023` to enable the latest **WebAssembly 2023** support.

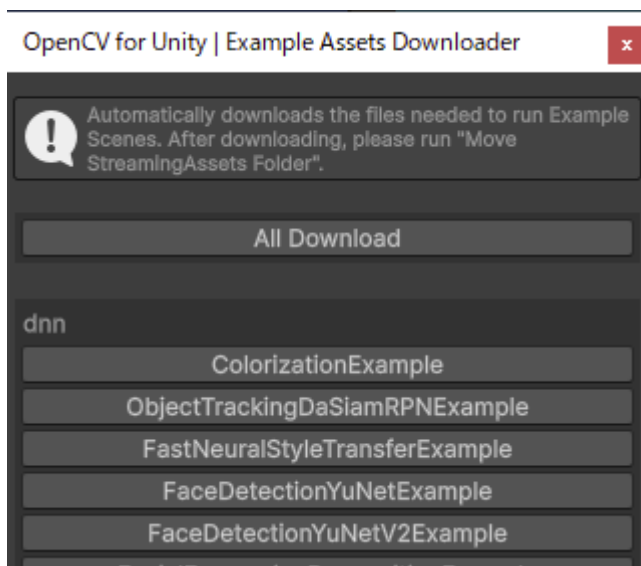
This setting automatically enables SIMD, WebAssembly native exceptions, BigInt, WebAssembly.Table, optimized data operations, and faster data type conversions.

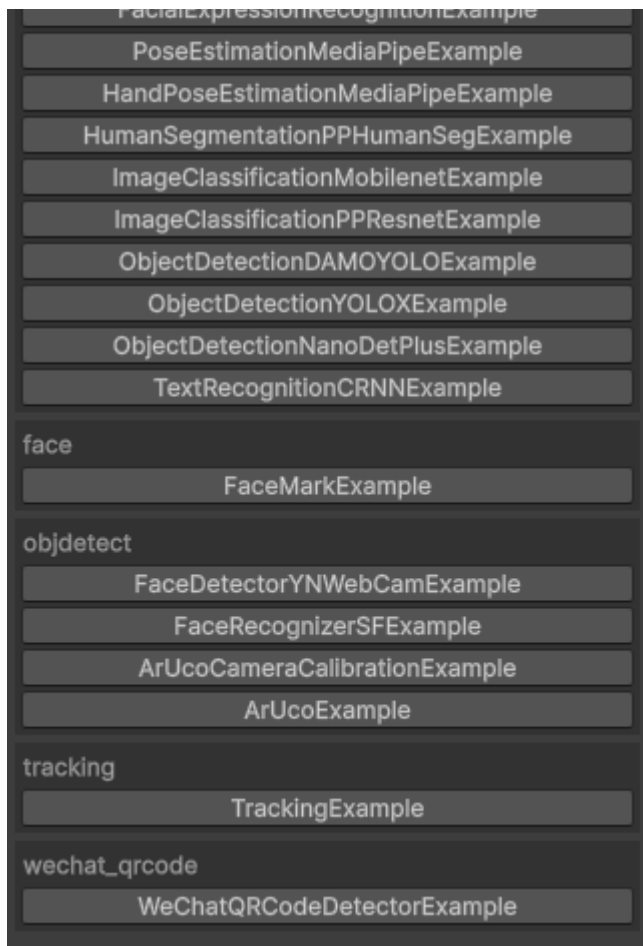
This setting is supported in Unity 2023.3 and later.

For details, please see [Unity Official Documentation](#).

Example Assets Downloader Window Description

Select the menu item `Tools > OpenCV for Unity > Open Example Assets Downloader` to open the `Example Assets Downloader` window. It can also be opened from the `Setup Tools` window.



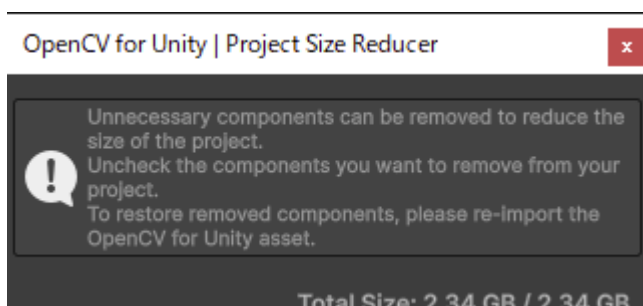


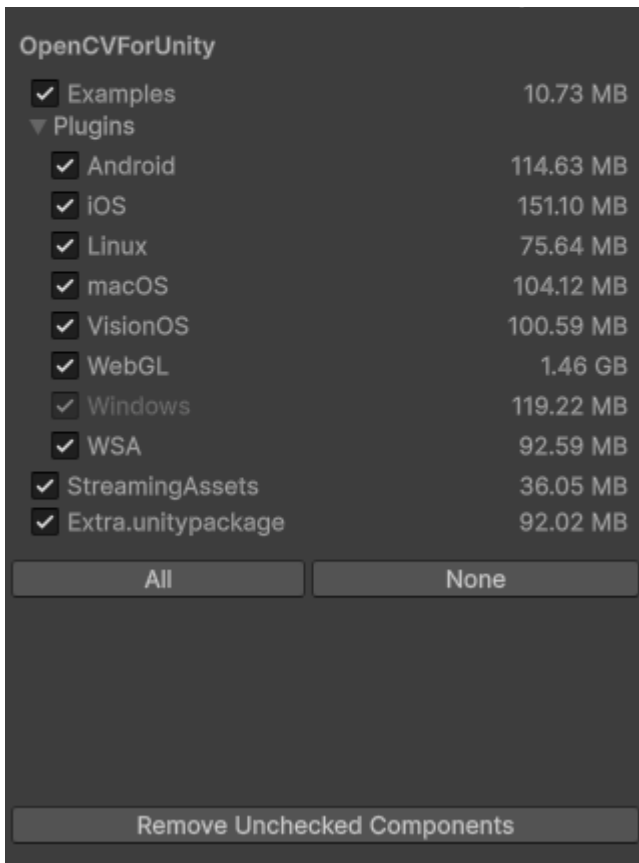
In this window, you can automatically download files necessary to run Example Scenes. The **All Download** button downloads files necessary for all Example Scenes. To download files necessary for specific scenes, click the respective Example Scene buttons. After downloading, please execute the **Move StreamingAssets Folder** button.

Tip: If you enabled **Enable Unity Sentis Integration** in step 3, **Unity Sentis** (`com.unity.ai.inference`) will **automatically** generate **.sentis** assets when **.onnx** model files from this download are imported into the project.

Project Size Reducer Window Description

Select the menu item **Tools > OpenCV for Unity > Open Project Size Reducer** to open the **Project Size Reducer** window. It can also be opened from the **Setup Tools** window.





In this window, you can remove unnecessary components to reduce project size. Uncheck the components you want to remove from the project and click the **Remove Unchecked Components** button. To restore deleted components, reimport the **OpenCV for Unity** asset.

Tips for Each Platform

• Android

- If you don't use the `opencv_contrib` module, you can reduce build size by using native plugin files that exclude the `opencv_contrib` module.
 - a. Click the **Import Extra Package** button to import the `Assets/OpenCVForUnity/Extra` folder.
 - b. Replace the `Assets/OpenCVForUnity/Plugins/Android/libs` folder with the `Assets/OpenCVForUnity/Extra/exclude_contrib/Android/libs` folder.
 - c. Click the **Set Plugin Import Settings** button.
 - d. Delete the `Assets/OpenCVForUnity/org/opencv_contrib` folder and `Assets/OpenCVForUnity/Examples/ContribModules` folder.

• iOS

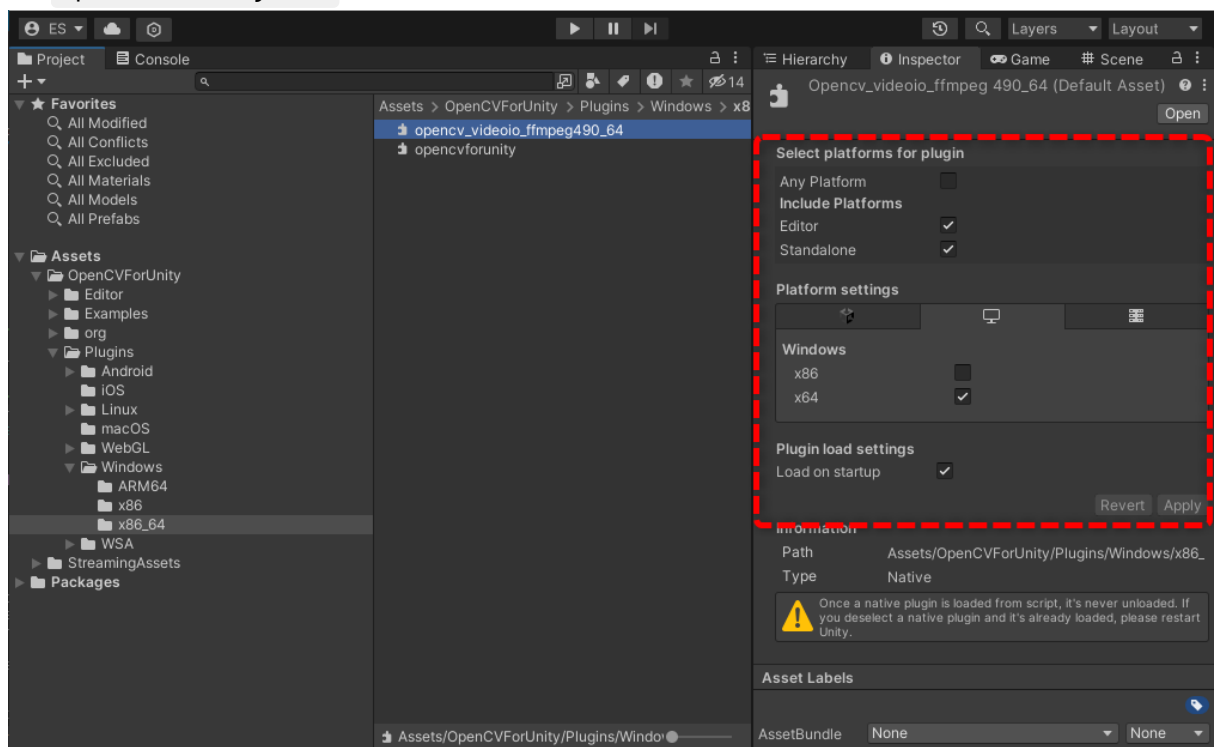
- If you don't use the `opencv_contrib` module, you can reduce build size by using native plugin files that exclude the `opencv_contrib` module.
 - a. Click the **Import Extra Package** button to import the `Assets/OpenCVForUnity/Extra`

folder.

- b. Replace the `Assets/OpenCVForUnity/Plugins/iOS` folder with the `Assets/OpenCVForUnity/Extra/exclude_contrib/iOS` folder.
- c. Click the `Set Plugin Import Settings` button.
- d. Delete the `Assets/OpenCVForUnity/org/opencv_contrib` folder and `Assets/OpenCVForUnity/Examples/ContribModules` folder.

- **Windows**

- If you want to use more video formats by specifying the ffmpeg backend in `Video Capture` (string filename) or `VideoWriter` methods, setup is required.
 - a. Download **OpenCV for Windows Version 5.0.0** (<http://opencv.org/downloads.html>).
 - b. Copy `opencv/build/bin/opencv_videoio_ffmpeg500_64.dll` to the `Assets/OpenCVForUnity/Plugins/Windows/x86_64` folder.
 - c. Set the `Import Settings` of `opencv_videoio_ffmpeg500_64.dll` to the same settings as `opencvforunity.dll` in the same folder.



You can use ffmpeg as the backend by setting `Videoio.CAP_FFMPEG` to `apiPreference` as shown in the following code.

The ffmpeg backend may allow you to specify more file formats than the default backend.

```
capture.open(Utils.getPath(VIDEO_FILENAME),  
Videoio.CAP_FFMPEG);  
Debug.Log("capture.getBackendName(): " + capture.getBackendName());  
  
writer.open(savePath, Videoio.CAP_FFMPEG, VideoWriter.fourcc('M', 'J',  
'P', 'G'), 30, new Size((int)captureRectPixel.width,
```

```
(int)captureRectPixel.height));  
Debug.Log("writer.getBackendName(): " + writer.getBackendName());
```

- **WebGL**

- If you want to minimize the app's build size, try the following steps.

- a. **Player Settings** settings related to build size optimization:

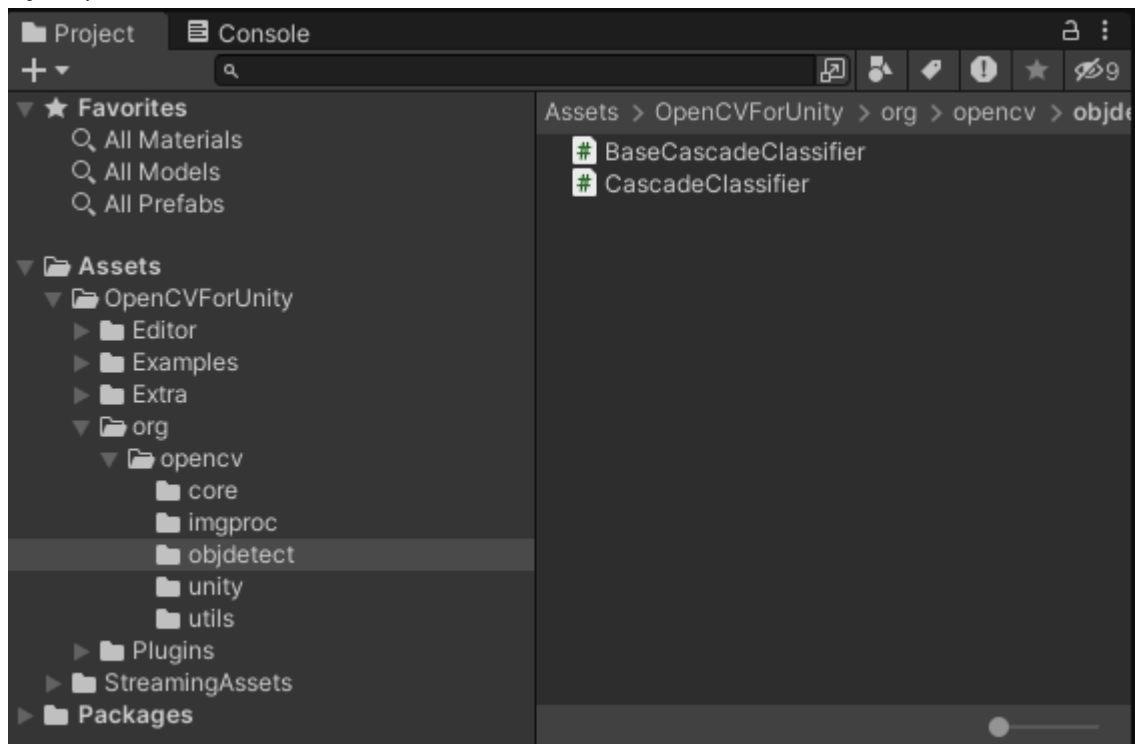
Code Optimization : Size , il2cppCodeGeneration : Faster (smaller) builds ,
managedStrippingLevel : High , DecompressionFormat : Brotli .

- b. Remove unused code from the **Assets/OpenCVForUnity/org** folder.

For example, the result when only the files necessary for **FaceDetectionExample** to work are left:

Code Optimization : Size , il2cppCodeGeneration : Faster (smaller) builds ,
managedStrippingLevel : High , DecompressionFormat : Brotli

→ Result: **6.31 MB** (6,622,861 bytes) → Build folder only result **5.41 MB** (5,681,838 bytes)



- **visionOS (beta)**

- On Vision Pro, even if you use **WebCamTexture** , you cannot obtain real-world camera images (pass-through images). Example Scenes using **WebCamTexture** will not work, so for applications that use camera functionality, you need to use Vision Pro-specific camera APIs.

How to use OpenCV Dynamic Link Library with customized build settings

 **Tip:** If you want faster **Dnn module** inference, open **Setup Tools** and click **Enable Unity Sentis Integration** (available on **Unity 6** or later) to try **GPU** inference through **Unity Sentis** (`com.unity.ai.inference`) using the **MultiBackendDnn** and **MultiBackendNet** workflow. For the full procedure, see step 3 in the [Example Scenes Setup Guide](#).

Download **OpenCV 5.0.0** repository (git: [opencv](#), [opencv-contrib](#)).

- **Android**

i. Change `self.cmake_vars['ANDROID_STL'] = 'c++_shared'` to `'c++_static'` in https://github.com/opencv/opencv/blob/4.x/platforms/android/build_sdk.py.

ii. Build the **Android SDK** with `"opencv/platforms/android/build_sdk.py" . ( APP_STL := c++_static )`

```
python ../opencv/platforms/android/build_sdk.py ../build ../opencv --  
ndk_path=C://android-ndk --sdk_path=C://android-sdk --  
extra_modules_path=../opencv_contrib/modules --config=ndk-18-api-  
level-24.config.py --no_samples_build --no_kotlin
```

iii. Copy the output file (`native\libs\arm64-v8a\libopencv_java5.so`) to `OpenCVForUnity\Plugins\Android\libs\arm64-v8a` folder.

Copy the output files (`native\libs\armeabi-v7a\libopencv_java5.so`) to `OpenCVForUnity\Plugins\Android\libs\armeabi-v7a` folder.

Copy the output files (`native\libs\x86_64\libopencv_java5.so`) to `OpenCVForUnity\Plugins\Android\libs\x86_64` folder.

iv. Copy `OpenCVForUnity\Extra\dll_version\Android\libs` folder to `OpenCVForUnity\Plugins\Android\libs` folder.

- **iOS**

i. Build the **iOS framework** with `"opencv/platforms/ios/build_framework.py" .`

```
python opencv/platforms/ios/build_framework.py --contrib opencv_contrib --  
dynamic ios
```

ii. Copy the output file (`opencv2.framework`) to `OpenCVForUnity\Plugins\iOS` folder.

- **Windows**

i. Build the **OpenCV dynamic library**.

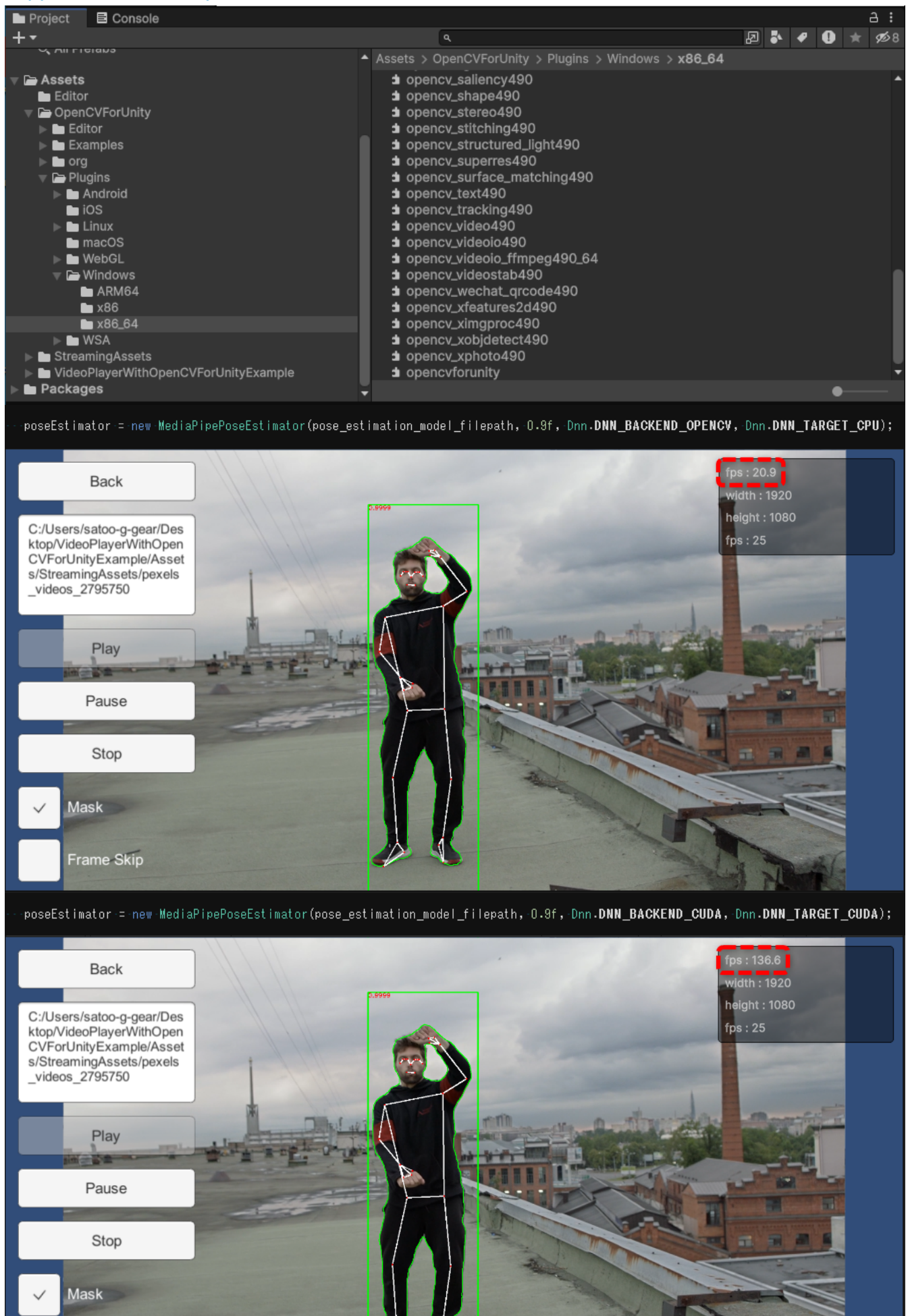
```
OPENCV_EXTRA_MODULES_PATH:PATH=C:/Users/xxxxx/opencv_contrib/  
modules BUILD_SHARED_LIBS:BOOL=ON
```

ii. Copy `install\x64\vc16\bin` to `OpenCVForUnity\Plugins\Windows\x64` folder.

Copy `OpenCVForUnity\Extra\dll_version\Windows` to `OpenCVForUnity\Plugins\Windows` folder.

Comparison of inference speed on Windows platform using the dnn module enabled the CUDA backend.

<https://enoxsoftware.com/opencvforunity/how-to-run-dnn-modules-with-cuda-backend-support-on-windows-platform/>





- macOS

- i. Build the **OpenCV** library.

```
CMAKE_OSX_ARCHITECTURESSTRING=x86_64  
OPENCV_EXTRA_MODULES_PATHPATH=/Users/xxxxx/opencv_contrib/  
modules
```

- ii. Copy the output files (`libopencv_*.5.0.0.dylib`) to
`opencvforunity.bundle\Contents\MacOS` folder.

- iii. Relink `libopencv_*.5.0.0.dylib` using `otool` and `install_name_tool` . <http://phenixyu.blogspot.com/2016/09/how-to-load-dynamic-library-with-unity.html>

```
//example : aruco module  
otool -L libopencv_aruco.5.0.0.dylib  
install_name_tool -id @loader_path/libopencv_aruco.5.0.0.dylib  
libopencv_aruco.5.0.0.dylib  
install_name_tool -change @rpath/libopencv_calib3d.5.0.dylib @loader_path/  
libopencv_calib3d.5.0.0.dylib libopencv_aruco.5.0.0.dylib  
install_name_tool -change @rpath/libopencv_features2d.5.0.dylib  
@loader_path/libopencv_features2d.5.0.0.dylib libopencv_aruco.5.0.0.dylib  
install_name_tool -change @rpath/libopencv_flann.5.0.dylib @loader_path/  
libopencv_flann.5.0.0.dylib libopencv_aruco.5.0.0.dylib  
install_name_tool -change @rpath/libopencv_highgui.5.0.dylib @loader_path/  
libopencv_highgui.5.0.0.dylib libopencv_aruco.5.0.0.dylib  
install_name_tool -change @rpath/libopencv_videoio.5.0.dylib @loader_path/  
libopencv_videoio.5.0.0.dylib libopencv_aruco.5.0.0.dylib  
install_name_tool -change @rpath/libopencv_imgcodecs.5.0.dylib  
@loader_path/libopencv_imgcodecs.5.0.0.dylib libopencv_aruco.5.0.0.dylib  
install_name_tool -change @rpath/libopencv_imgproc.5.0.dylib @loader_path/  
libopencv_imgproc.5.0.0.dylib libopencv_aruco.5.0.0.dylib  
install_name_tool -change @rpath/libopencv_core.5.0.dylib @loader_path/  
libopencv_core.5.0.0.dylib libopencv_aruco.5.0.0.dylib
```

- Linux

- i. Build the **OpenCV** library.
- ii. Rename output files (`libopencv_*.so.5.0.0`).

```
sudo apt-get install rpl  
sed -i -e 's/.so.500/500.so/g' ./libopencv.so.5.0.0*  
rename "s/.so.5.0.0/500.so/" libopencv.so.5.0.0*  
find . -type f -name '_500.so' -print -exec patchelf --set-rpath '$ORIGIN' {} ;*
```

iii. Copy `libopencv_xxxxx_500.so` to `OpenCVForUnity\Plugins\Linux\x86_64` folder.

- **UWP**

i. Build the **OpenCV library**.

```
cd C:/Users/satoo/Desktop/opencv/platforms/winrt
setup_winrt.bat "WS" "10.0" "x64"
```

ii. Copy `install/WS/10.0/ARM/ARM/vc16/bin` folder to

`Assets/OpenCVForUnity/Plugins/WSA/UWP/ARM` folder. Copy

`install/WS/10.0/x64/x64/vc16/bin` folder to `Assets/OpenCVForUnity/Plugins/WSA/UWP/x64`

folder. Copy `install/WS/10.0/x86/x86/vc16/bin` folder to

`Assets/OpenCVForUnity/Plugins/WSA/UWP/x86` folder.

Migrating Code from 3.0.3 to the Latest Version

If you developed your project using **3.0.3**, you can update to the latest version (**3.0.4**) by following the steps below. Starting with **3.0.4**, the scripts under `Assets/OpenCVForUnity/Scripts` have been reorganized into **Core**, **Extensions**, and **UnityIntegration** layers. The old API remains available for compatibility in `Assets/OpenCVForUnity/Scripts/UnityIntegration/Compatibility`, but we recommend migrating to the new API to avoid **Obsolete** warnings.

1. Update the Asset Package

1. Back up your project.
2. Delete the existing `Assets/OpenCVForUnity` folder.
3. Import the latest **OpenCV for Unity** from the `Package Manager`.
4. For details, please see [Setup Guide for Getting Started](#).

2. Choose a Migration Approach

You can choose either of the following approaches.

Option A: Keep Using the Compatibility Layer (Short-Term)

This approach requires no code changes and is suitable when you want to update the asset quickly.

- Existing code that uses APIs in

`Assets/OpenCVForUnity/Scripts/UnityIntegration/Compatibility` (for example `OpenCvEnv`, `OpenCvMatUtils`, `OpenCvDebug`, `*2MatHelper`, `Vec*`, `MOT`, `Runner`, or `Worker` classes) continues to compile.

- **Obsolete** warnings are displayed in the Unity Console.
- Compatibility classes are located in `Assets/OpenCVForUnity/Scripts/UnityIntegration/Compatibility` .

Note: The Compatibility layer is provided for migration purposes. We recommend migrating to the new API as soon as possible.

Option B: Update to the New API (Recommended)

This approach removes **Obsolete** warnings and lets you use the latest API.

Update your `using` directives and API calls as described in the sections below.

3. Update Namespaces for Unity Integration APIs

Starting with **3.0.4**, scripts under `Assets/OpenCVForUnity/Scripts` are reorganized into **Core**, **Extensions**, and **UnityIntegration** layers. APIs that existed in **3.0.3** are retained in `Assets/OpenCVForUnity/Scripts/UnityIntegration/Compatibility` for backward compatibility but are marked as deprecated. The tables below summarize how namespaces changed for each area.

Layer	Folder	Role
Core	Assets/ OpenCVForUnity/ Scripts/ Core	OpenCV native bindings and core Mat APIs
Extensions	Assets/ OpenCVForUnity/ Scripts/ Extensions	Pure C# extension APIs (no MonoBehaviour)
UnityIntegration	Assets/ OpenCVForUnity/ Scripts/ UnityIntegration	Unity-facing helpers, workers, and UI components

Namespace transition summary by area

Area	Namespace (3.0.3)	Namespace (latest version)	Notes
Unity integration utilities	OpenCVForUnity. UnityIntegration	OpenCVForUnity. UnityIntegration or OpenCVForUnity. Extensions	Class names also changed for some APIs (see below)
Vec types (Vec2d, Vec3f, etc.)	OpenCVForUnity. UnityIntegration	OpenCVForUnity. Extensions	Same struct names

Area	Namespace (3.0.3)	Namespace (latest version)	Notes
Source2Mat helper MonoBehaviour classes	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	OpenCVForUnity. UnityIntegration. Helper. SourceToMat	Class names also changed (see section 4)
Source2Mat core types and utilities	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	OpenCVForUnity. Extensions. SourceToMat	Interfaces, enums, and utility classes
MOT (multi-object tracking)	OpenCVForUnity. UnityIntegration. MOT	OpenCVForUnity. Extensions. MOT	Includes ByteTrack sub-namespace
Async runner utilities	OpenCVForUnity. UnityIntegration. Runner	OpenCVForUnity. Extensions. Runner	Same class names
Worker base classes and utilities	OpenCVForUnity. UnityIntegration. Worker	OpenCVForUnity. Extensions. Worker	Includes Utils and DataStruct sub-namespaces
DNN inference worker base classes	OpenCVForUnity. UnityIntegration. Worker. Dnn	OpenCVForUnity. Extensions. Worker. DnnModule	Dnn renamed to DnnModule
DNN MultiBackend facades	OpenCVForUnity. UnityIntegration. Worker. Dnn	OpenCVForUnity. UnityIntegration. Worker. DnnModule	Remain in UnityIntegration (namespace segment renamed)
OpenCV extension overloads (Span / Struct / ValueTuple)	OpenCVForUnity. *Module	OpenCVForUnity. *Module	Namespace unchanged; implementations moved to Extensions/ org

You can confirm the current API in the [API Reference](#).

Unity integration utilities

Class (3.0.3)	Namespace (3.0.3)	Class (latest version)	Namespace (latest version)
---------------	-------------------	------------------------	----------------------------

Class (3.0.3)	Namespace (3.0.3)	Class (latest version)	Namespace (latest version)
OpenCVEnv	OpenCVForUnity. UnityIntegration	OpenCVForUnityEnv	OpenCVForUnity. UnityIntegration
OpenCVDebug	OpenCVForUnity. UnityIntegration	OpenCVDebug	OpenCVForUnity. Extensions
OpenCVMat Utils (Mat ↔ Unity conversion)	OpenCVForUnity. UnityIntegration	OpenCVMatUnityUtils	OpenCVForUnity. UnityIntegration
OpenCVMat Utils (Mat buffer copy)	OpenCVForUnity. UnityIntegration	MatBuffer Utils	OpenCVForUnity. Extensions

Vec types

Class (3.0.3)	Namespace (3.0.3)	Class (latest version)	Namespace (latest version)
Vec2b– Vec10d	OpenCVForUnity. UnityIntegration	Vec2b–Vec10d	OpenCVForUnity. Extensions

Source2Mat helpers and core types

Class / type (3.0.3)	Namespace (3.0.3)	Class / type (latest version)	Namespace (latest version)
WebCamTexture2Mat Helper, MultiSource2Mat Helper, etc.	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	WebCamTextureToMat Helper, MultiSourceToMat Helper, etc.	OpenCVForUnity. UnityIntegration. Helper. SourceToMat
ISource2Mat Helper	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	IMatSource	OpenCVForUnity. Extensions. SourceToMat
ICameraSource2Mat Helper	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	ICameraMatSource	OpenCVForUnity. Extensions. SourceToMat

Class / type (3.0.3)	Namespace (3.0.3)	Class / type (latest version)	Namespace (latest version)
ImageSource2MatHelper	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	ImageFileMatSource	OpenCVForUnity. Extensions. SourceToMat
IVideoSource2MatHelper	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	IVideoFileMatSource, IVideoCaptureMatSource	OpenCVForUnity. Extensions. SourceToMat
ITextureSource2MatHelper	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	IUnityTextureMatSource	OpenCVForUnity. Extensions. SourceToMat
IMatTransformationProvider	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	IMatTransformable	OpenCVForUnity. Extensions. SourceToMat
IMatUpdateFPSProvider	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	IMatUpdateFPSControllable	OpenCVForUnity. Extensions. SourceToMat
Source2MatHelper ColorFormat	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	SourceToMat ColorFormat	OpenCVForUnity. Extensions. SourceToMat
Source2MatHelper ErrorCode	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	SourceToMat ErrorCode	OpenCVForUnity. Extensions. SourceToMat
Source2MatHelper ErrorUnityEvent	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	SourceToMat ErrorUnityEvent	OpenCVForUnity. UnityIntegration. Helper. SourceToMat

Class / type (3.0.3)	Namespace (3.0.3)	Class / type (latest version)	Namespace (latest version)
Source2MatHelper Utils	OpenCVForUnity. UnityIntegration. Helper. Source2Mat	SourceToMat Utils	OpenCVForUnity. Extensions. SourceToMat

MOT (multi-object tracking)

Class (3.0.3)	Namespace (3.0.3)	Class (latest version)	Namespace (latest version)
IMultiObjectTracker	OpenCVForUnity. UnityIntegration. MOT	IMultiObjectTracker< T >	OpenCVForUnity. Extensions. MOT
ITrackInfoVisualizer	OpenCVForUnity. UnityIntegration. MOT	ITrackInfoVisualizer< T >	OpenCVForUnity. Extensions. MOT
TrackInfo, BBox	OpenCVForUnity. UnityIntegration. MOT	TrackInfo, BBox	OpenCVForUnity. Extensions. MOT
TrackInfoVisualizer	OpenCVForUnity. UnityIntegration. MOT	TrackInfoVisualizer	OpenCVForUnity. Extensions. MOT
BYTETracker, BYTETrackInfo	OpenCVForUnity. UnityIntegration. MOT. ByteTrack	BYTETracker, BYTETrackInfo	OpenCVForUnity. Extensions. MOT. ByteTrack

Runner

Class (3.0.3)	Namespace (3.0.3)	Class (latest version)	Namespace (latest version)
WorkCompletionKind, WorkCompletion< TResult >	OpenCVForUnity. UnityIntegration. Runner	WorkCompletionKind, WorkCompletion< TResult >	OpenCVForUnity. Extensions. Runner
SingleFlightSyncAsync Runner< TInput, TResult	OpenCVForUnity. UnityIntegration.	SingleFlightSyncAsync Runner< TInput, TResult	OpenCVForUnity. Extensions.

Class (3.0.3)	Namespace (3.0.3)	Class (latest version)	Namespace (latest version)
>	Runner	>	Runner
MatSingleFlightSyncAsync Runner	OpenCVForUnity. UnityIntegration. Runner	MatSingleFlightSyncAsync Runner	OpenCVForUnity. Extensions. Runner

Worker (processing and DNN)

Class (3.0.3)	Namespace (3.0.3)	Class (latest version)	Namespace (latest version)
IProcessingWorker	OpenCVForUnity. UnityIntegration. Worker	IProcessingWorker	OpenCVForUnity. Extensions. Worker
ProcessingWorker Base	OpenCVForUnity. UnityIntegration. Worker	ProcessingWorker Base	OpenCVForUnity. Extensions. Worker
ClassLabel Utils, MatPool	OpenCVForUnity. UnityIntegration. Worker. Utils	ClassLabel Utils, MatPool	OpenCVForUnity. Extensions. Worker. Utils
DnnProcessing Utils	OpenCVForUnity. UnityIntegration. Worker. Utils	DnnNms Utils	OpenCVForUnity. Extensions. Worker. Utils
ObjectDetectionData, ClassificationData, etc.	OpenCVForUnity. UnityIntegration. Worker. DataStruct	ObjectDetectionData, ClassificationData, etc.	OpenCVForUnity. Extensions. Worker. DataStruct
DnnInferenceWorker Base	OpenCVForUnity. UnityIntegration. Worker. Dnn	DnnInferenceWorker Base	OpenCVForUnity. Extensions. Worker. DnnModule
YOLOv8ObjectDetector MultiBackend, etc.	OpenCVForUnity. UnityIntegration. Worker. Dnn	YOLOv8ObjectDetector MultiBackend, etc.	OpenCVForUnity. UnityIntegration. Worker. DnnModule

Note: DNN inference worker base classes move to

OpenCVForUnity.Extensions.Worker.DnnModule . MultiBackend facade classes remain in OpenCVForUnity.UnityIntegration.Worker.DnnModule but the namespace segment Dnn is renamed to DnnModule .

OpenCV extension overloads (Span / Struct / ValueTuple)

Compatibility overloads for OpenCV classes (for example `MatOfPoint` Span / Struct constructors, `Imgproc` Struct overloads) remain in the same OpenCV module namespace (for example `OpenCVForUnity.CoreModule` , `OpenCVForUnity.ImgprocModule`). In the latest version, the recommended implementations are provided under `Assets/OpenCVForUnity/Scripts/Extensions/org` . Update deprecated overloads to the corresponding `OpenCVForUnity.Extensions` API when **Obsolete** warnings appear.

Before (3.0.3):

```
using OpenCVForUnity.UnityIntegration;
using OpenCVForUnity.UnityIntegration.Helper.Source2Mat;
using OpenCVForUnity.UnityIntegration.MOT;

string filePath = await
OpenCVEnv.GetFilePathTaskAsync("OpenCVForUnityExamples/768x576_rgb.png");
OpenCVDebug.SetDebugMode(true);
OpenCVMatUtils.Texture2DToMat(texture2D, mat);
WebCamTexture2MatHelper webCamHelper = GetComponent();
// your OpenCV calls here
OpenCVDebug.SetDebugMode(false);
```

After (latest version):

```
using OpenCVForUnity.UnityIntegration;
using OpenCVForUnity.UnityIntegration.Helper.SourceToMat;
using OpenCVForUnity.Extensions;
using OpenCVForUnity.Extensions.SourceToMat;
using OpenCVForUnity.Extensions.MOT;

string filePath = await
OpenCVForUnityEnv.GetFilePathAsync("OpenCVForUnityExamples/768x576_rgb.png")
;
OpenCVDebug.SetDebugMode(true);
OpenCVMatUnityUtils.Texture2DToMat(texture2D, mat);
WebCamTextureToMatHelper webCamHelper = GetComponent();
// your OpenCV calls here
```

```
OpenCVDebug.SetDebugMode(false);
```

For details on catching native OpenCV error codes, please see [Q9](#).

4. Update Source2Mat Helper Classes

For namespace changes of Source2Mat-related APIs, please see [section 3](#). If your **3.0.3** code uses `*2MatHelper` classes in `OpenCVForUnity.UnityIntegration.Helper.Source2Mat`, update the class names and namespace to `OpenCVForUnity.UnityIntegration.Helper.SourceToMat`.

Old Helper (3.0.3)	New Helper (latest version)
WebCamTexture2Mat Helper	WebCamTextureToMat Helper
WebCamTexture2MatAsyncGPU Helper	WebCamTextureToMat Helper (<i>use RequestedUseAsyncGPUReadback for AsyncGPU</i>)
MultiSource2Mat Helper	MultiSourceToMat Helper
Image2Mat Helper	ImageFileToMat Helper
VideoCapture2Mat Helper	VideoCaptureToMat Helper
AsyncGPUReadback2Mat Helper	AsyncGPUReadbackToMat Helper
UnityVideoPlayer2Mat Helper	UnityVideoPlayerToMat Helper

Before (3.0.3):

```
using OpenCVForUnity.UnityIntegration.Helper.Source2Mat;  
  
WebCamTexture2MatHelper webCamHelper = GetComponent();  
Mat rgbaMat = webCamHelper.GetMat();
```

After (latest version):

```
using OpenCVForUnity.UnityIntegration.Helper.SourceToMat;  
  
WebCamTextureToMatHelper webCamHelper = GetComponent();  
Mat rgbaMat = webCamHelper.FrameMat;
```

Note: In the latest version, subscribe to `SourceToMatHelperBase.OnFrameMatUpdated` for frame updates instead of polling `GetMat()` every frame.

Example (latest version):

```
using OpenCVForUnity.CoreModule;
```

```

using OpenCVForUnity.ImgcodecsModule;
using OpenCVForUnity.UnityIntegration;

string filePath = await
OpenCVForUnityEnv.GetFilePathAsync("OpenCVForUnityExamples/768x576_rgb.png")
;
using (var mat = Imgcodecs.imread(filePath))
{
    OpenCVMatUnityUtils.MatToTexture2D(mat, texture2D);
}

```

For complete working examples, please see the `WebCamTextureToMatHelperExample` and `MultiSourceToMatHelperExample` scenes in the `Examples` folder.

5. Update OpenCV Module Namespaces (OpenCV 4 to OpenCV 5)

The latest version is based on **OpenCV 5.0.0**. **OpenCV 5** reorganized several modules that were previously provided as a single module in **OpenCV 4**. If your **3.0.3** code uses **OpenCV 4**-based module names, update your `using` directives and class references according to the table below.

OpenCV 4	OpenCV 5	Description	OpenCV for Unity Namespace
calib3d	geometry	2D / 3D / nD geometry processing	OpenCVForUnity. GeometryModule
calib3d	calib	Camera calibration	OpenCVForUnity. CalibModule
calib3d	stereo	Stereo depth estimation	OpenCVForUnity. StereoModule
calib3d	ptcloud	High-level 3D processing	OpenCVForUnity. PtcloudModule
features2d	features	Module rename and extended functionality	OpenCVForUnity. FeaturesModule
ml	opencv_contrib	Classic ML moved to contrib	OpenCVForUnity. MIModule
objdetect	objdetect + xobjdetect	Haar / HOG detectors partially	OpenCVForUnity. ObjdetectModule,

OpenCV 4	OpenCV 5	Description	OpenCV for Unity Namespace
		moved	OpenCVForUnity. XobjdetectModule

*Note: The former `calib3d` module is split across multiple modules in **OpenCV 5**. Identify which new module contains the API you use before updating your code.*

You can confirm the current API in the [API Reference](#).

`calib3d` **split (geometry / calib / stereo / ptcloud):**

Before (OpenCV 4):

```
using OpenCVForUnity.Calib3dModule;

Calib3d.solvePnP(objectPoints, imagePoints, camMatrix, distCoeffs, rvec, tvec);
Calib3d.calibrateCamera(objectPoints, imagePoints, imageSize, cameraMatrix,
distCoeffs, rvecs, tvecs, flags);
```

After (OpenCV 5):

```
using OpenCVForUnity.GeometryModule;
using OpenCVForUnity.CalibModule;

Geometry.solvePnP(objectPoints, imagePoints, camMatrix, distCoeffs, rvec, tvec);
Calib.calibrateCamera(objectPoints, imagePoints, imageSize, cameraMatrix, distCoeffs,
rvecs, tvecs, flags);
```

`features2d` → `features` :

Before (OpenCV 4):

```
using OpenCVForUnity.Features2dModule;

ORB orb = ORB.create();
```

After (OpenCV 5):

```
using OpenCVForUnity.FeaturesModule;

ORB orb = ORB.create();
```

`objdetect` + `xobjdetect` **(Haar / HOG detectors):**

Before (OpenCV 4):

```
using OpenCVForUnity.ObjdetectModule;

HOGDescriptor hog = new HOGDescriptor();
CascadeClassifier cascade = new CascadeClassifier();
```

After (OpenCV 5):

```
using OpenCVForUnity.XobjdetectModule;

HOGDescriptor hog = new HOGDescriptor();
CascadeClassifier cascade = new CascadeClassifier();
```

For **Classic ML** APIs formerly in the `m1` module, continue using `OpenCVForUnity.M1Module`. The implementation is now provided under `opencv_contrib`.

6. Verify Your Migration

1. Open your project in Unity and confirm that script compilation completes without errors.
2. If you chose Option A, check the Console for **Obsolete** warnings and plan to address them.
3. Run your scenes on target platforms and verify image processing and display.

You can confirm the current API in the [API Reference](#).

? Q & A

Q1. The asset package size is large. Is there a way to reduce it?

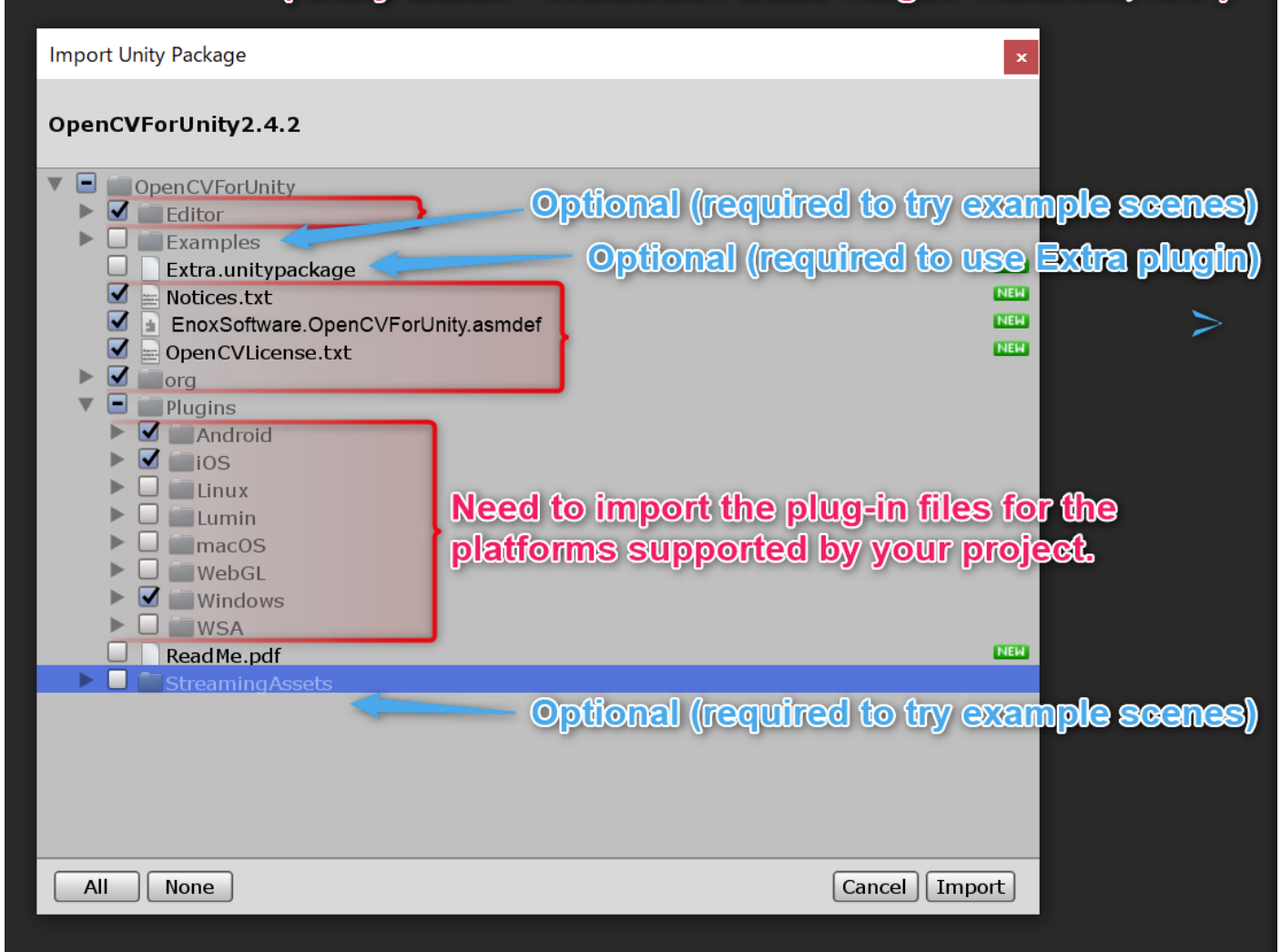
A1. Using the `Project Size Reducer`, you can easily reduce unnecessary files in the `Assets/OpenCVForUnity` folder. Particularly, by removing plugin files for platforms not planned to be supported in your development project, you can significantly reduce the project size. For details, please see [Example Assets Downloader Window Description](#).

Q2. What is the minimum file configuration required for the asset to work?

A2. You don't need to import all files for the asset to work. If you don't need to try Example Scenes, the minimum file configuration required is as follows.

Example Of Minimum File Composition:

(Unity Editor - Windows / Build Target - Android, iOS)



Q3. How can I learn the details of **OpenCV** methods and arguments?

A3. For details on method arguments, please refer to [OpenCV Official Documentation](#) and [OpenCV Tutorials](#).

Q4. How can I convert `Mat` class operators defined in **C++**?

A4. Regarding the conversion method of `Mat` class operators defined in **C++**. For details, please see this page. ([Way to translation of Mat class operators defined in C++](#))

Q5. When running Example Scenes, `DllNotFoundException: opencvforunity` appears in the console.

A5. It seems the plugin is not loaded correctly. Please check the setup procedure.

Q6. When running Example Scenes, `ArgumentException: The output Mat object has to be of the same`

appears in the console.

A6. After plugin setup, restarting the **Unity Editor** may make the plugin work properly.

Q7. When running Example Scenes,

Level 'Texture2DtoMatExample' (-1) could not be loaded because appears in the console.

A7. Click the `Add Example Scenes in Build` button in the `Setup Tools` window to add all Example Scene files in the `Examples` folder to `Scenes In Build` in `Build Settings`.

Q8. How can I use **SIFT** and **SURF** algorithms?

A8. The native libraries included in **OpenCV for Unity** are built with the `OPENCV_ENABLE_NONFREE` flag disabled. To use **SIFT** and **SURF** algorithms, please rebuild the OpenCV library with `OPENCV_ENABLE_NONFREE` enabled. For details, please see [How to use OpenCV Dynamic Link Library with customized build settings](#).

Q9. How to catch error codes that occur on the native side of OpenCV (**CVException** handling)

A9. To display error codes from the native side of OpenCV, wrap your code with `OpenCVDebug.SetDebugMode(true)` and `OpenCVDebug.SetDebugMode(false)`.

```
using OpenCVForUnity.Extensions;
```

```
OpenCVDebug.SetDebugMode(true);
```

```
// your OpenCV calls here
```

```
OpenCVDebug.SetDebugMode(false);
```

For details, please see this page. ([How to catch native OpenCV's errors code \(CVException handling\)](#))

Version Changes

3.0.4 [Common] Updated OpenCV to version 5.0.0. [Common] Reorganized scripts into Core, Extensions, and UnityIntegration structure. [Common] Updated Mat ptr/at/AsSpan/get/put APIs and MatOf*/Converters for OpenCV 5 0D/1D Mat shape compatibility. [Common] Migrated Examples folder structure to OpenCV 5.0.0 module layout. [Common] Migrated to

SourceToMatHelper system with MultiSourceToMatHelper and SourceToMatControlPanel; deprecated legacy Source2Mat helpers. [Common] Redesigned MultiBackendDnn as OpenCV/Sentis separation facade and added DNN Worker Extensions/UnityIntegration separation. [Common] Added YOLOv5/v8 detection, classification, segmentation, and pose estimation Examples. [Common] Added Sentis inference backend support to ColorizationExample, FastNeuralStyleTransferExample, and ImageClassificationMobilenetExample. [Common] Split OpenCVMatUtils into OpenCVMatUnityUtils and MatBufferUtils with compatibility facade. [Common] Renamed OpenCVEnv to OpenCVForUnityEnv. [Common] Renamed async APIs from *TaskAsync to *Async and Awaitable APIs to *AwaitableAsync with obsolete aliases. [Common] Added FaceIdentificationEstimatorExample face Save/Load feature. [Common] Added DerivedFrame layout event integration to MultiSourceToMatHelper. [Common] Updated ARHelper-related classes, added ARPoseEstimator to Extensions.AR.

3.0.3 [Common] Added MultiBackendDnn and MultiBackendNet classes for easily switching between the Unity Sentis GPU inference backend and the OpenCV Dnn backend. Extensively updated the Dnn module examples accordingly. [Common] Added MediaPipeFaceLandmarkerExample, MediaPipeHandLandmarkerExample, MediaPipePoseLandmarkerExample and MediaPipeHolisticLandmarkerExample. [WebGL] Added Unity6000.5 or later support. [Common] Updated ARHelper-related classes.

3.0.2 [Common] Updated OpenCV to version 4.13.0. [Common] Changed TextureSelector class to use PointerEvents for implementation. [Common] WebCamTexture2MatHelper does not work with WebGPU, so use WebCamTexture2MatAsyncGPUHelper instead for WebGPU. [Common] Changed ImwriteScreenCaptureExample scene to use ScreenCapture.CaptureScreenshotAsTexture method to work with both Built-in and SRP.

3.0.1 [Common] Changed the minimum supported version to Unity2021.3.45f2. [Common] Added BallTrackingBasedOnColorExample, HumanPoseStreamEstimationMediaPipeExample, FaceIdentificationEstimatorExample. [Common] Added FaceIdentificationEstimator class for face detection, tracking, and recognition pipeline with BYTETracker integration for robust multi-object tracking. [Common] Added MediaPipePoseStreamEstimator class for continuous pose tracking in video streams using MediaPipePersonDetector and MediaPipePoseEstimator with temporal smoothing. [Common] Added MediaPipeHandPoseStreamEstimator class for continuous hand tracking in video streams using MediaPipePalmDetector and MediaPipeHandPoseEstimator with temporal smoothing. [Common] Updated ARHelper-related classes.

3.0.0 [Common] Updated OpenCV to version 4.12.0. [Common] Updated FpsMonitor to version 1.0.4. [visionOS] Updated platform support for visionOS on Unity 2022.3.18f1 and later (beta). [Android] Updated native library libopencvforunity.so for Android to be compatible with 16KB page size. [Common] Added SafeDrawFrameAxes method to OpenCVARUtils. [Common] Added GetFilePathCoroutine, GetFilePathAsync, and GetFilePathTaskAsync methods to OpenCVEnv. [Common] Added IsDebugMode and IsThrowException methods to OpenCVDebug. [Common] Added TextureToMatAsync, RenderTextureToMatAsync, and MatToRenderTextureAsync methods to OpenCVMatUtils. [Common] Added OpenCVForUnityProjectSizeReducerWindow in

EditorWindow. [Common] Added a feature to skip unnecessary downloads in OpenCVForUnityExampleAssetsDownloaderWindow. [Common] Added ImshowDNNBlob method to DebugMat. [Common] Added WeChatQRCodeDetectorImageExample and WeChatQRCodeDetectorExample. [Common] Added HomographyToFindAKnownObjectExample. [Common] Added ObjectDetectionDAMOYOLOExample. [Common] Removed FaceDetectionResnetSSDExample due to incompatibility with the latest OpenCV. [Common] Changed namespaces under org/unity from UnityUtils to UnityIntegration, split Utils class into function-specific classes, and marked old classes as deprecated. [Common] Changed Operator in Mat class to deprecated.

2.6.6 [Common] Changed The FpsMonitor display system from IMGUI to uGUI. [Common] Added support for the new Input System (UnityEngine.InputSystem).

2.6.5 [Common] Add ARHelper component class. [Common] Updated ArUcoExample, ArUcoImageExample, HandPoseEstimationMediaPipeExample and PostEstimationMediaPipeExample using the ARHelper class. [Common] Refactored AsyncGPUReadback2MatHelper, Image2MatHelper, UnityVideoPlayer2MatHelper, VideoCapture2MatHelper, WebCamTexture2MatAsyncGPUHelper and WebCamTexture2MatHelper. [Common] Added method to convert OpenCV basic classes to Unity basic structs.

2.6.4 [Common] Added various data access methods to the Mat class, including ptr(), at(), AsSpan(), and AsSpanRowRange(). [Common] Updated get() and set() methods in the Mat class to support Span and generics, and updated MatUtils.copyFromMat() and MatUtils.copyToMat() to support Span. [Common] Added structs and ValueTuples corresponding to the data formats of basic OpenCV classes, and created overloads for methods that use these basic classes as arguments, allowing the use of structs and ValueTuples instead. This update simplifies code, enables more efficient memory usage, and improves data access speed. [Common] Added Utils.getPathAsyncTask() and Utils.getMultipleFilePathsAsyncTask() methods. It is an asynchronous Task return version of the existing Utils.getPathAsync() method. It can be seamlessly integrated with other asynchronous code. [Common] Added Utils.texture2DToMatRaw() and Utils.matToTexture2DRaw() methods. [Common] Added methods for conversion between Mat and RenderTexture using ComputeShader (Utils.matToRenderTexture(), Utils.renderTextureToMat()). [Common] Added the WebCamTexture2MatAsyncGPUHelper class. [Common] Updated the DebugMatUtils class to support displaying Texture2D and RenderTexture. [Common] Added a face blurring feature to FaceDetectorYNetExample. [iOS] Fixed an issue where AddToEmbeddedBinaries was processed redundantly with each incremental build.

2.6.3 [Common] Fixed CountFingersExample. [Windows][UWP] Fixed native library build settings for ARM64 architecture.

2.6.2 [Common] Changed the minimum supported version to Unity2021.3.35f1. [Common] Added AsyncGPUReadback2MatHelper. [Common] Updated MultiSource2MatHelperExample. [Common] Separated the examples using the Built-in Render Pipeline and Scriptable Render

Pipeline. [Common]Fixed a memory leak in Converters.cs.

2.6.1 [Common]Updated to OpenCV4.10.0. [Common]Added MultiSource2MatHelperExample.

2.6.0 [iOS] Added separate plugin files for iOS for devices and simulators. [WebGL] Added plugin files with only simd enabled.

2.5.9 [Common]Added PoseSkeletonVisualizer to HandPoseEstimationMediaPipeExample and PoseEstimationMediaPipeExample. [Common]Changed to use unsafe code by default.

[Common]Optimized the amount of memory allocation, mainly in the Convertor class.

2.5.8 [Common]Updated to OpenCV4.9.0. [Common]Added DebugMatUtilsExample and MultiObjectTrackingExample. [Common]Updated VideoWriterExample, VideoWriterAsyncExample, TextRecognitionCRNNExample, TextRecognitionCRNNWebCamExample and TrackingExample.

[Common]Changed the minimum supported version to Unity2020.3.48f1. [WebGL] Added support for "WebAssembly 2023". [iOS] Changed "Target minimum iOS Version" to 11.0.

[Common]Removed TrackerGOTURN from TrackingExample and added TrackerVit.

2.5.7 [Common]Added FeatureMatchingExample. [WebGL] Added a plugin file with threads and simd enabled for the WebGL platform. This update removes support for the WebGL platform in Unity 2021.1 and below. (Select MenuItem[Tools/OpenCV for Unity/Open Setup Tools/WebGL Settings])

2.5.6 [Common]Added PoseEstimationMediaPipeExample. [Common]Updated VideoWriterExample, VideoWriterAsyncExample (support URP and HDRP).

2.5.5 [Common]Updated to OpenCV4.8.0. [Common]Added ImageCorrectionExample, YuNetFaceDetectionExample and FaceDetectionYuNetV2Example. [Common]Updated HumanSegmentationExample, BarcodeDetectorExample. [Windows] Added Support for ARM64. [WebGL] Added Unity2023.2 or later support.

2.5.4 [Common]Added VideoWriterAsyncExample. [Common]Updated SimpleBlobExample, HumanSegmentationExample and HandPoseEstimationExample.

2.5.3 [Common]Added HandPoseEstimationExample and FacialExpressionRecognitionExample.

2.5.2 [Common]Added YOLOv4ObjectDetectionExample, YOLOv7ObjectDetectionExample, YOLOXObjectDetectionExample and NanoDetPlusObjectDetectionExample. [Common]Updated HandPoseEstimationExample, ArUcoExample, ArUcoWebCamExample and ArUcoCameraCalibrationExample.

2.5.1 [Common]Updated to OpenCV4.7.0. [Common]Added ImageClassificationMobilenetExample and HandPoseEstimationExample. [Common]Added TrackerNano to TrackingExample. [Lumin] Removed Lumin platform support (for MagicLeapOne). [Common]Add a button to SetupTools to automatically add scenes under the "Examples" folder to "Scenes In Build".

2.5.0 [Common]Added TransformECCEExample, HumanSegmentationExample and ImageClassificationPPResnetExample. [Common]Update TextOCRWebCamExample.

[Common]Changed the setup procedure to use the SetupToolsWindow. [Common]Change the namespace under "OpenCVForUnity/Editor" folder from "OpenCVForUnity" to

"OpenCVForUnity.Editor". [Common]Added the ExampleAssetsDownloaderWindow that

automatically downloads the necessary files to Examples. [Common]Added "OpenCVForUnity"

folder under "StreamingAssets" folder. [Common]Added function to automatically move the StreamingAssets folder. [WebGL] Added Unity2022.2 or later support.

2.4.9 [Common]Added LegacyTrackingExample and LightweightPoseEstimationWebCamExample. [Common]Renamed "WebCamTextureExample" to "WebCamExample". [WebGL] Fixed plugins for the WebGL platform.

2.4.8 [Common]Updated to OpenCV4.6.0. [Common]Added KNNExample, PhysicalGreenScreenExample and LightweightPoseEstimationExample. [Common]Update VideoCaptureToMatHelper, GreenScreenExample and QRCodeDetectorExample.

2.4.7 [Common]Updated to OpenCV4.5.5. [Common]Added FaceDetectorYNWebCamTextureExample, FaceRecognizerSFExample and QRCodeEncoderExample.

2.4.6 [Common]Updated to OpenCV4.5.4. [Android] Added Support for ChromeOS (x86 and x86_64 architectures).

2.4.5 [Common]Updated to OpenCV4.5.3. [Common]Added BarcodeDetectorExample and BarcodeDetectorWebCamTextureExample.

2.4.4 [Common]Updated to OpenCV4.5.2. [Common]Added VideoCaptureCameraInputExample and BackgroundSubtractorComparisonExample. [Common]Updated TrackingExample.

2.4.3 [Common]Added a downloader script to automatically set up the Dnn module example. [WebGL] Added exclude_contrib version for build size reduction.

2.4.2 [Common]Added Assembly Definitions. [Common]Fixed LibFaceDetectionV3Example.

2.4.1 [Common]Updated to OpenCV4.5.0. [Common]Added DaSiamRPNTrackerExample.

2.4.0 [Common]Updated to OpenCV4.4.0. [Common]Added TextOCRExample. [Common]Updated YoloObjectDetectionExample (Yolo v4).

2.3.9 [Common]Updated to OpenCV4.3.0. [Common]Added LibFaceDetectionV2Example, LibFaceDetectionV3Example, ColorizationExample and DocumentScannerExample. [Common]Update ArUcoCameraCalibrationExample and WrapPerspectiveExample.

2.3.8 [Common]Updated to OpenCV4.2.0. [UWP] Added ARM64 Architecture. [WebGL] Added opencvforunity.bc with multi-threading enabled. [Common]Added FastNeuralStyleTransferExample and LibFaceDetectionExample. [Common]Added MatIndexer class and MatUtils class. [Common]Update ComicFilterExample, VideoCaptureExample, OpenPoseExample and MatBasicProcessingExample.

2.3.7 [WebGL] Fixed build errors that occur when DevelopmentBuild is enabled on the WebGL platform. [Common]Added optimization code using NativeArray class. (require PlayerSettings.allowUnsafeCode flag, "OPENCV_USE_UNSAFE_CODE" ScriptingDefineSymbol and Unity2018.2 or later.) [iOS] Fixed build errors that occur on the iOS platform with Unity2019.3 or later. [Common]Updated to WebCamTextureToMatHelper.cs v1.1.1.

2.3.6 [WebGL] Fixed "Plugins/WebGL/2018.2/opencvforunity.bc". [Common]Added multi-dimensional Mat example to MatBasicProcessingExample. [Common]Fixed ARUtils.cs.

2.3.5 [Common]Updated to OpenCV4.1.0. [Windows, Android] Added dynamic link library version.

2.3.4 [Common]Added MaskRCNNExample. [WebGL] Added Unity2019.1 or later support.

2.0.9 [WebGL] Added WebGL(beta) support.(Unity5.3 or later)

2.0.8 [Common]Improved WebCamTextureHelper class. [Common]Fixed ArUcoSample.

2.0.7 [Common]Added aruco, structured_light, xfeatures2d module. [Common]Added ArUcoSample, GrabCutSample, InpaintSample, MatchShapesSample, MSERSample.

2.0.6 [WSA] Fixed an issue where Windows App Certification Kit fails.

2.0.5 [Common]Added HOGDescriptorSample.

2.0.4 [Android] Added Support for Split Application Binary (.OBB) [Android] Removed opencvforunity.jar.

2.0.3 [Common]Added SVMSample. [Common]Fixed VideoCaptureSample and WebCamTextureAsyncDetectFaceSample. [UWP] Added OpenCVForUnityUWP_Beta2.zip.

2.0.2 [Common]Fixed CS0618 warnings:

`UnityEngine.Application.LoadLevel(string)' is obsolete: Use SceneManager.LoadScene'.`

2.0.1 [OSX] Fixed SIGILL Exception. [Common]Added Utils.setDebugMode() method. [Common]Added MatchTemplateSample, StereoBMSample, SeamlessCloneSample and WebCamTextureDetectCirclesSample. [Common]Added flipVertical flag, flipHorizontal flag and GetWebCamDevice() method to WebCamTextureToMatHelper.cs.

2.0.0 [Common]Updated to OpenCV3.1.0. [Common]Included Old Version based on "OpenCV2.4.11". [Common] Included Beta Version of Windows10 UWP Support.(This is beta version based on OpenCV3.0.0. opencv_contrib modules is not supported.)

Beta16 [iOS] Fixed libopencvforunity.a Bitcode Setting.

Beta15 [Common]Fixed WebCamTextureToMatHelper.cs.(Add didUpdateThisFrame () method)

Beta14 [Common]Fixed WebCamTextureToMatHelper.cs.(Bug of rotation conversion from WebCamTexture to Mat in Win,Mac StandAlone Build)

Beta13 [Common]Added fastTexture2DToMat() and fastMatToTexture2D(). [Common] Renewed the samples using WebCamTextureToMatHelper.(Supports all screen orientation.)

Beta12 [iOS] Fixed malloc_error that occurs in Unity5.3.1p2.

Beta11 [iOS] Enabled Jpeg format.(Added mjpeg format support in VideoCapture class)

Beta10 [iOS] Enabled Bitcode.

Beta9 [UWP] Added support for Windows10 UWP.(This is a test version. opencv_contrib modules is not supported.)

Beta8 [Common] Fix FaceRecognizerSample. [Common] Delete the method using Default parameter specifiers. [Android] Compile the library using "armabi-v7a with NEON" option.

Beta7 [Common] Add WrapPerspectiveSample, HandPoseEstimationSample.

Beta6 [iOS] Fix WebCamTexture bug of SampleScene in Unity5.2.

Beta5 [Linux] Add Linux Support. [WindowsStoreApp8.1] Support for methods using Low-level Native Plugin Interface. [Common] Rewrite SampleScene.

Beta4 [Common] Add Utils. getGraphicsDeviceType(). [Common] Add SampleScene Setup Tutorial Video for Unity5.

Beta3 [Common] Add CamShiftSample.(Object Tracking) [Common] Add OpenCVForUnityMenuItem.cs.(This script set plugin import settings automatically from

MenuItem.)

Beta2 [iOS] Fix problem when working with Metaio(UnityAppController problem). [Common]Add [System.Serializable] to basic class. [Common] change folder name from "OpenCVForUnity/OpenCVForUnity_Editor/" to "OpenCVForUnity/Editor/". [iOS] Move "OpenCVForUnity/OpenCVForUnity_Editor/opencv2.framework" to "OpenCVForUnity/Plugins/iOS" folder.